

The Physics of Multi-Turn Long-Horizon Planning: From Pre-training to Post-training via Single and Multi Teachers On-Policy Agentic Distillation

Tianyi Men^{1, 1}

¹CASIA

Multi-turn long-horizon planning is a critical ability for foundation model agents, but how to fundamentally improve this ability across different training stages remains unclear. Existing foundation models are trained on uncontrollable and opaque Internet data, making it difficult to systematically analyze how long-horizon planning ability is acquired and improved across different training stages. In this work, we introduce a controlled multi-turn long-horizon planning environment that enables precise control over task length, planning difficulty, data quality, planning knowledge, and planning patterns to systematically study long-horizon planning improvement across three training stages: pre-training, single-teacher post-training (GRPO and OPD), and multi-teacher model merging. Through extensive experiments, we obtain several findings. **(1) In the pre-training stage**, we analyze the impact of training data from three perspectives: data format, distribution, and quality. Chain-of-thought based state transition modeling improves long-horizon generalization over direct action prediction. While atomic skills alone cannot achieve long-horizon planning, adding a small amount of long-horizon data effectively improves ability. Moreover, suboptimal trajectories severely harm planning due to accumulated decision errors. **(2) In the single-teacher post-training stage**, we find that planning ability consists of two aspects from the perspective of mutual information: planning patterns and planning knowledge. Planning patterns represent general behaviors shared across tasks, while planning knowledge contains specific procedures and task-dependent solutions. For planning patterns, on-policy distillation has a broader effective boundary than RL under low-quality data and long-horizon tasks, as sparse credit assignment leads to noisy RL updates. However, for planning knowledge, distilling unseen multi-path solutions can damage the student's existing world model when teacher and student follow different procedures, as distillation may overwrite old knowledge without fully acquiring new knowledge. **(3) In the multi-teacher model merging stage**, we reveal that capability transfer depends on shared structures among expert planning patterns. Compatible patterns enable cross-environment generalization, while partially shared conflicting patterns support continual learning by balancing new knowledge and forgetting. However, completely conflicting patterns without shared structures cause catastrophic forgetting and domain interference. Our study provides a systematic understanding of how long-horizon planning ability is acquired, shaped, and integrated across training stages, offering insights for developing stronger agentic foundation models towards multi-turn ultra-long horizon agentic era.

Keywords: Multi-turn, Long-Horizon, Agentic, Planning, Pre-training, RL, OPD, MOPD

Homepage Link: <https://quester-one.github.io/PlanPhysWebsite/>

Github Link: <https://github.com/Quester-one/PlanPhysCode>

Huggingface Model: https://huggingface.co/MultimodalAgent/TianyiMen_PlanPhys_Models

Huggingface Dataset: https://huggingface.co/datasets/MultimodalAgent/TianyiMen_PlanPhys_Datasets

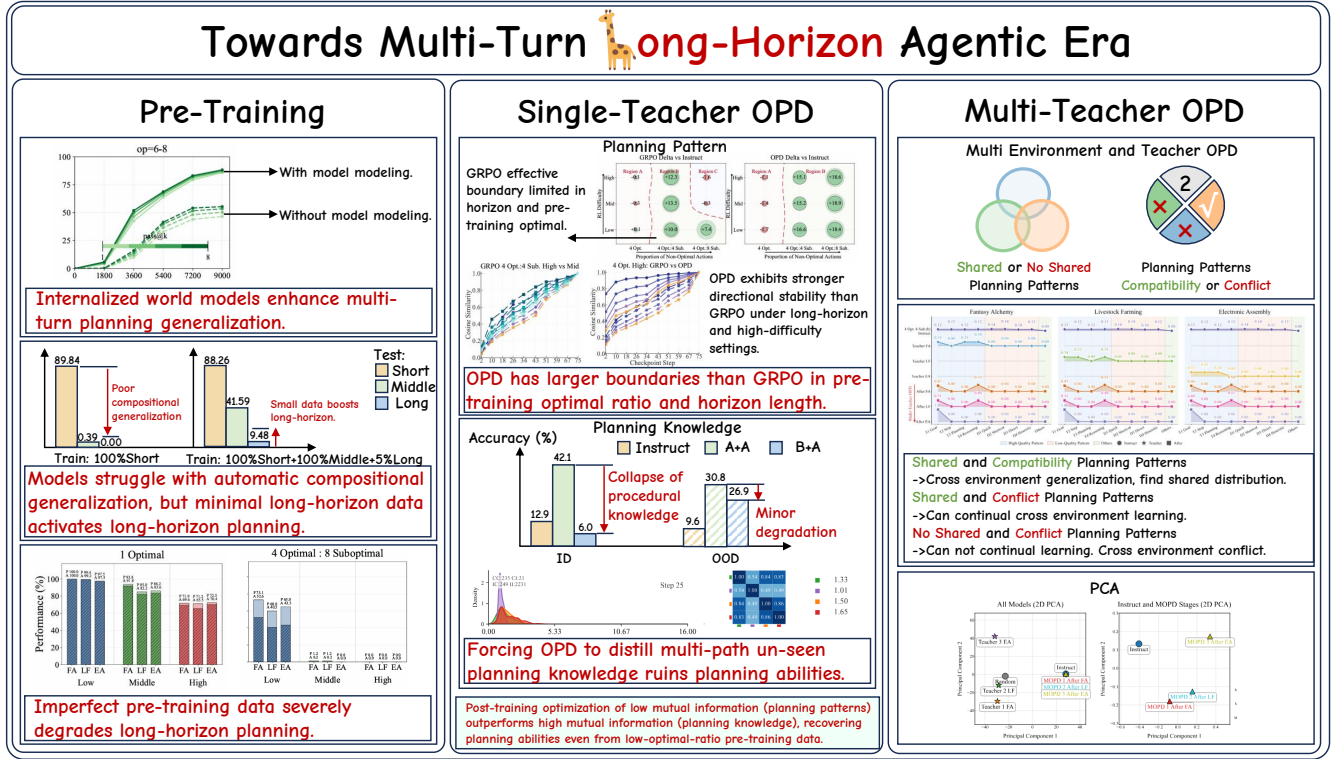


Figure 1: Overview of multi-turn long-horizon agentic experimental results. We select the giraffe to represent the 'L' in 'Long', as its L-shaped neck and broad horizon view align with the long-horizon goal.

1. Introduction

As foundation models advance toward agentic systems, **multi-turn long-horizon planning** ability becomes a critical capability for building real-world general intelligent agents (Yuan et al., 2026, Zhu et al., 2026a, Li et al., 2026f, Zhang et al., 2026b). Different from single-turn question answering or short-sequence decision making, long-horizon planning requires agents to decompose complex tasks, track environment states, model state transitions, and compose atomic skills through long-horizon interactions (Li et al., 2026b, Cao et al., 2025). Recently, benchmarks for multi-turn long-horizon planning, including OSWorld 2.0 (Yuan et al., 2026), EdgeBench (Zhu et al., 2026a), Long-Horizon-Terminal-Bench (Li et al., 2026f), and DeepPlanning (Zhang et al., 2026b), demonstrate that foundation models achieve basic short-horizon planning capabilities, but they still struggle with long-horizon planning tasks.

Currently, improving the long-horizon planning ability of agents mainly relies on three stages: (1) **Large-scale pre-training** to learn general planning knowledge and planning patterns, (2) **RL-based post-training** to further optimize long-horizon planning strategies, and (3) **Multi-teacher model merging** in post-training to combine long-horizon planning abilities from multiple teachers. However, one key question remains unclear:

Fundamental Question for Multi-Turn Long-Horizon Planning.

How to fundamentally improve long-horizon planning ability of foundation models in different training stages?

However, understanding and improving the long-horizon planning ability of foundation models still faces a fundamental challenge. The gap is that existing foundation models rely on **uncontrollable and opaque Internet data**, making it difficult to systematically analyze where planning ability comes from and how it can be further strengthened. For example, it remains unclear whether the long-horizon planning ability exhibited by models already exists during the pre-training stage or whether it is a new ability activated during the post-training stage that does not exist in pre-training. Therefore, when we aim to further improve the long-horizon planning ability of models, we cannot answer the following three basic questions from the perspective of the training stages:

Fundamental Sub-question for Multi-Turn Long-Horizon Planning.

- (1) Pre-training Stage.** *How does the pre-training data acquire long-horizon planning through world model internalization, atomic skill composition, and trajectory quality?*
- (2) Single-Teacher Post-training Stage.** *How does RL shape long-horizon planning, and what are the applicable boundaries of RL algorithms in shaping planning knowledge and patterns?*
- (3) Multi-Teacher Model Merging Stage.** *How does model merging integrate long-horizon planning capabilities to enable cross-environment generalization, continual learning, and teacher conflict resolution?*

To answer the above question, we propose a **unified and controlled multi-turn environment to study long-horizon planning ability**. The environment allows precise control over the number of environments, planning levels, task lengths, data quality, planning knowledge, and planning patterns. It supports both imitation learning and reinforcement learning paradigms.

Based on this framework, we draw the following conclusions through three progressive stages, as shown in Figure 1:

Pre-training Stage. In the pre-training stage, we analyze the acquisition of long-horizon planning ability from **three data perspectives: data format, data distribution, and data quality**. From the perspective of data format, the model explicitly constructs an internal world model through chain-of-thought for state transition modeling, which leads to stronger generalization ability in long-horizon planning compared with direct action prediction. From the perspective of data distribution, although it is difficult for the model to achieve compositional generalization only through atomic skills, adding a small amount of long-horizon data during pre-training can enhance the long-horizon planning ability. From the perspective of data quality, real-world pre-training data inevitably contains suboptimal trajectories rather than only optimal ones. Since errors in long-sequence decision making can accumulate and amplify over time, such imperfect data can severely damage long-horizon planning ability.

Single-Teacher Post-training Stage. In the single-teacher post-training stage, we view planning ability from **the perspective of mutual information and divide it into two aspects: planning patterns and planning knowledge**. The key difference between them is that planning patterns capture general planning behaviors shared across tasks, while planning knowledge contains specific procedures and task-dependent solutions. From the perspective of planning patterns, under the two dimensions of planning horizon and planning quality, GRPO has a narrower application boundary. When the pre-training data quality is low and the agent requires long-horizon planning, OPD exhibits a broader effective boundary for general planning patterns. This is because sparse credit assignment in RL causes gradient updates to contain both correct and incorrect directions, while OPD provides more stable and consistent update directions. From the perspective of planning knowledge, the situation is different. When the teacher and student models have different underlying procedural planning knowledge, directly applying OPD to distill unseen multi-path knowledge cannot guarantee effective transfer, even if the teacher achieves high accuracy. Instead, it may damage the student’s existing in-domain world modeling and reduce out-of-domain planning ability. This is because specific planning knowledge contains higher mutual information and requires precise updates for individual cases. However, RL only provides small parameter updates, which are insufficient to fully replace old knowledge with new unseen knowledge. Therefore, the model may enter an intermediate state where existing knowledge is damaged but new planning knowledge is not fully established.

Multi-Teacher Model Merging Stage. In the multi-teacher model merging stage, we analyze **cross-environment generalization, continual learning, and cross-environment conflicts from two aspects: whether different planning patterns are shared or compatible across different environments**. The experiments show that the key mechanism of sequential merging is to find and converge to the shared distribution among multiple teacher policies. When experts from different environments share compatible planning patterns, multi-teacher on-policy distillation (MOPD) can break the boundaries between domains and successfully generalize the abilities activated in one environment to unseen environments. When facing conflicting teacher patterns, as long as there are still some shared and useful pattern structures among different domains, the model can learn new knowledge while avoiding severe forgetting, which supports continual learning. However, when experts from different domains have no shared planning patterns and completely conflict with each other, the student model suffers from severe catastrophic forgetting during overfitting to the new expert. This leads to interference and loss of cross-environment abilities.

2. Preliminaries

2.1. Agent Planning

Planning Definition. In this work, we formulate agent planning as a sequential decision-making problem. Given an initial environmental state $s_0 \in \mathcal{S}$ and a specific target goal $g \in \mathcal{G}$, the objective of the agent is to generate a sequence of executable actions $A = \{a_1, a_2, \dots, a_T\}$. The execution of this sequence transitions the environment from the initial state s_0 to a final state s_T that successfully satisfies the goal g .

Agent Skill/Experience/Procedural Knowledge/Script Knowledge. We define an agent skill (or experience or procedural knowledge or script knowledge) as a natural language mapping from a task description to a sequential list of sub-steps. Formally, a skill k in the skill library $\mathcal{K}$ is represented as a tuple $k = (d, P)$. Here, d represents the natural language description of the target task (e.g., "make a cup of coffee"), and $P = \langle p_1, p_2, \dots, p_m \rangle$ is an ordered sequence of sub-steps, where each step p_i is also explicitly described in natural language (e.g., "find a mug", "pour water"). This format provides the agent with a clear, semantic recipe detailing exactly how a specific sub-goal should be systematically achieved. In practice, these skills can be utilized in two primary paradigms: either explicitly provided as external context, or implicitly internalized into the model's chain-of-thought (CoT) reasoning process. In this paper, we primarily focus on investigating the latter approach.

World Model. The world model aims to learn the rules of state transitions. Specifically, it models the state transition function $T(s_t, a_t) \rightarrow s_{t+1}$, which determines the next state given the current state and a specific action. In this process, procedural skills are not treated as separate, external modules; instead, they are directly internalized into the state transition function itself. By doing so, the model embeds these skills as the underlying rules driving state changes, enabling it to predict future states based on given actions.

2.2. Agent On-Policy Distillation

Agent On-Policy Distillation (Agent-OPD) optimizes the student policy π_θ directly on its self-explored interactive trajectories. Formally, we define a K -turn trajectory $\tau = (s_0, g, \hat{Y}_1, s_1, \dots, \hat{Y}_K, s_K) \sim \pi_\theta$, where $s_0 \in \mathcal{S}$ and $g \in \mathcal{G}$ are the initial state and goal. At each turn k , conditioned on the accumulated context H_k , the student generates a unified sequence $\hat{Y}_k = (\hat{y}_{k,1}, \dots, \hat{y}_{k,T_k})$ that sequentially interleaves intermediate planning with an executable action, subsequently transitioning the environment to s_k .

At each token $t \in \{1, \dots, T_k\}$ within turn k , both the student and a teacher policy π_{teacher} evaluate the sequence prefix to yield next-token distributions:

$$p_{k,t} \triangleq \pi_\theta(\cdot \mid H_k, \hat{Y}_{k,<t}) \quad \text{and} \quad q_{k,t} \triangleq \pi_{\text{teacher}}(\cdot \mid H_k, \hat{Y}_{k,<t}). \quad (1)$$

By autoregressively factorizing the reverse Kullback-Leibler (KL) divergence over the entire multi-turn rollout, the Agent-OPD objective admits an exact token-level decomposition. This provides dense, step-by-step supervision on both the reasoning chain and the decision-making process:

$$\mathcal{L}_{\text{Agent-OPD}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=1}^K \sum_{t=1}^{T_k} D_{\text{KL}}(p_{k,t} \parallel q_{k,t}) \right]. \quad (2)$$

PG-Style OPD vs GKD-Style OPD. Agent On-Policy Distillation primarily diverges into two paradigms based on how teacher feedback is utilized. Policy Gradient (PG) style OPD treats the per-token reverse KL divergence as a dense reward signal. Specifically, it computes the token-level advantage using a single-sample log-ratio approximation (Schulman K1): $A_{k,t} \approx \log p_{k,t}(\hat{y}_{k,t}) - \log q_{k,t}(\hat{y}_{k,t})$. The student is then updated via RL objectives to re-weight trajectory sampling probabilities: $\mathcal{L}_{\text{PG}} = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_{k,t} \text{sg}(A_{k,t}) \log p_{k,t}(\hat{y}_{k,t})]$, where $\text{sg}(\cdot)$ denotes the stop-gradient operator. In contrast, Generalized Knowledge Distillation (GKD) style OPD does not rely on sampled scalar advantages. Instead, it explicitly minimizes the statistical divergence between the full predictive distributions: $\mathcal{L}_{\text{GKD}} = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_{k,t} D_{\text{KL}}(p_{k,t} \parallel q_{k,t})]$. Consequently, while PG-style indirectly optimizes the policy by increasing the likelihood of trajectories with lower reverse KL, GKD-style provides explicit supervision to directly reshape the token-level distributions. Crucially, GKD-style ignores the global trajectory-sampling probability; instead, it leverages direct gradient propagation to minimize the local divergence between the student and teacher distributions exclusively on the already visited prefixes. In practice, Sampled-Token OPD predominantly adopts the PG and GKD paradigm, whereas Top- k and Full-Vocabulary OPD generally employ the GKD approach.

Sampled-Token OPD. Sampled-Token OPD evaluates the objective exclusively on the discrete tokens $\hat{y}_{k,t}$ sampled during the student’s rollout, thereby avoiding the computation of the full vocabulary distribution. This approach is typically implemented in two ways. The **PG-Style** formulation utilizes a policy gradient estimator, treating the log-ratio as a token-level reward: $r_{k,t} = \log p_{k,t}(\hat{y}_{k,t}) - \log q_{k,t}(\hat{y}_{k,t})$. Alternatively, the **GKD-Style** formulation directly minimizes a local distance metric on the sampled tokens. Two common mathematical forms for the GKD-Style objective are based on Schulman’s approximations: the **K1 approximation**, formulated as $\log p_{k,t}(\hat{y}_{k,t}) - \log q_{k,t}(\hat{y}_{k,t})$, and the **K2 approximation**, formulated as $\frac{1}{2}(\log p_{k,t}(\hat{y}_{k,t}) - \log q_{k,t}(\hat{y}_{k,t}))^2$.

Top-k OPD. Top- k OPD limits the divergence penalty to a truncated vocabulary space. Specifically, at each decoding step t within turn k , we isolate the subset of k tokens that are most probable according to the student policy $p_{k,t}$. Let us denote this high-confidence token subset as $\mathcal{V}_{k,t} \subset \mathcal{V}$. We then construct truncated and re-normalized probability mass functions for both the student and the teacher, restricted strictly to $\mathcal{V}_{k,t}$:

$$\tilde{p}_{k,t}(v) = \frac{p_{k,t}(v)}{\sum_{w \in \mathcal{V}_{k,t}} p_{k,t}(w)}, \quad \tilde{q}_{k,t}(v) = \frac{q_{k,t}(v)}{\sum_{w \in \mathcal{V}_{k,t}} q_{k,t}(w)}, \quad \forall v \in \mathcal{V}_{k,t}. \quad (3)$$

The distillation objective is subsequently formulated by evaluating the local KL divergence exclusively over these adjusted distributions. Integrating this into our agent’s multi-turn trajectory framework yields the following formulation:

$$\mathcal{L}_{\text{Top-}k}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=1}^K \sum_{t=1}^{T_k} D_{\text{KL}}(\tilde{p}_{k,t} \parallel \tilde{q}_{k,t}) \right]. \quad (4)$$

Full-Vocabulary OPD. In contrast to sampled or truncated approximations, Full-Vocabulary OPD evaluates the divergence penalty across the entire vocabulary $\mathcal{V}$ without any masking. At each decoding step t within turn k , it directly aligns the complete predictive distribution of the student with that of the teacher. The distillation objective function computes the exact Kullback-Leibler divergence over all possible tokens:

$$\mathcal{L}_{\text{Full-Vocab}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=1}^K \sum_{t=1}^{T_k} D_{\text{KL}}(p_{k,t} \parallel q_{k,t}) \right]. \quad (5)$$

3. Task Formulation

3.1. Controllable Planning Gym

Skill Graph Construction. We construct hierarchical skill graphs across three distinct domains (e.g., *Fantasy Alchemy*). To ensure rich semantic relationships, we leverage a large language model to generate category trees and concrete item entities. Each domain is structured as a tree with a height of H layers, where each layer encompasses W categories, and each category contains N unique concrete items.

The abstract graph transitions (i.e., synthesis recipes) systematically connect nodes from lower layers to higher layers. **Crucially, the synthesis rules governing these transitions are formulated as a logical combination of AND/OR operations over prerequisite items.** For a target node v , its valid synthesis pathways are defined by an **OR** relation over multiple independent recipes, while each individual recipe requires the simultaneous availability (**AND**) of its constituent ingredients. Formally, the synthesis logic for node v is expressed as:

$$v \iff (u_{i,1} \wedge \dots \wedge u_{i,k_i}) \vee \dots \vee (u_{m,1} \wedge \dots \wedge u_{m,k_m}) \quad (6)$$

where each conjunction block $(u_{i,1} \wedge \dots \wedge u_{i,k_i})$ represents a valid abstract recipe. To prevent high-step-count explosion, we employ a greedy minimization algorithm that prioritizes combining items with fewer preceding synthesis steps. For each domain, we generate disjoint abstract transition graphs (e.g., Graph A and Graph B). The transitions require varying numbers of prerequisite nodes governed by a fixed probability p , ensuring structurally sound and non-overlapping rule sets.

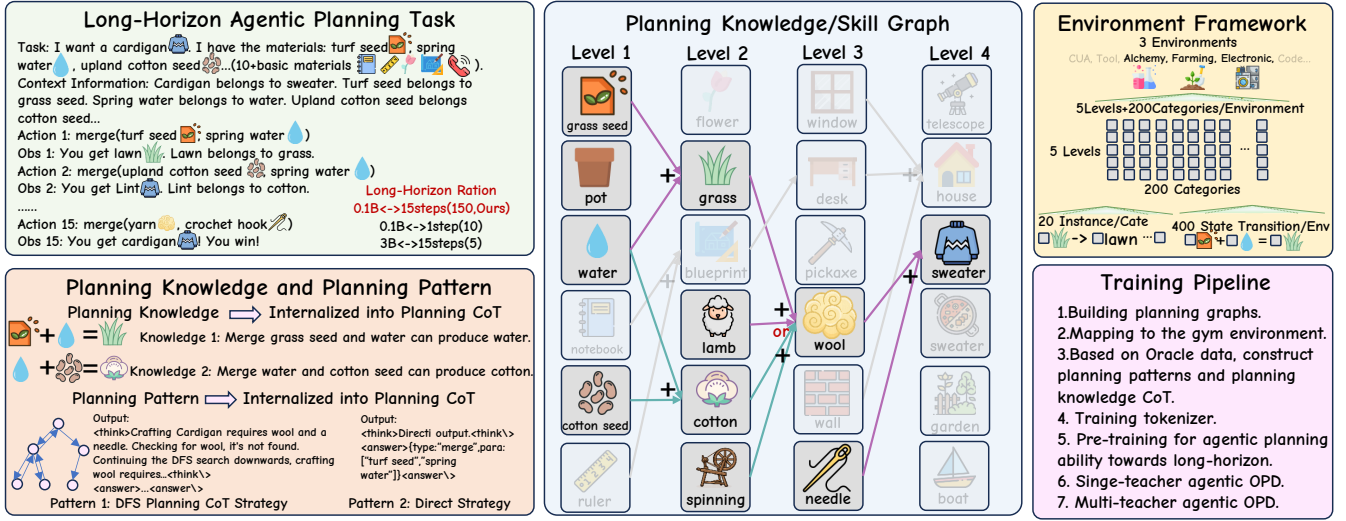


Figure 2: Overview of the proposed long-horizon agentic planning framework.

Graph-to-Gym Mapping. The abstract skill graphs are instantiated into concrete simulated environments (Gyms) by mapping abstract nodes to specific items via an *index map*. To foster environment diversity, we generate a combination of fixed and randomized configurations. Crucially, a subset of K item indices per category is strictly reserved for test set configurations. The remaining indices are randomly sampled to construct diverse training environments, preventing data leakage and ensuring evaluation on unseen entity mappings.

Task Difficulty Control. Task difficulty is determined by the minimum number of actions (i.e., synthesis steps) required to produce a target item from a given initial inventory. By sampling the target item from higher layers and starting materials from lower layers, we dynamically compute the exact step count required. Based on this, tasks are categorized into three difficulty levels: *Low*, *Mid*, and *High* step thresholds. To simulate realistic planning noise, we inject distractor items from the same starting layers into the initial inventory.

Dataset Splitting. To evaluate generalization, we split the instantiated configurations into datasets for pre-training, post-training, and testing. The test set exclusively employs novel combinations of familiar items. The training set is divided: the majority of configurations are utilized for pre-training to encourage exploratory learning across all trajectories, while a smaller subset is reserved for post-training, providing high-quality trajectories.

3.2. Task Setup

We formulate the planning problem as a sequential decision-making task. The agent is initialized with a starting inventory (prerequisites and distractors) and must synthesize a target item. The state observation includes natural language descriptions of the category each item belongs to, facilitating category-level reasoning without exposing underlying IDs. At each step, the model must select the correct sequence of materials from the inventory to execute valid synthesis actions until the target item is successfully created.

3.3. Evaluation

For evaluation, we uniformly sample M tasks across different step counts to ensure a balanced assessment spanning the difficulty spectrum. The evaluation is conducted on the reserved test configurations of Graph A for each difficulty level (*Low*, *Mid*, and *High*). The primary evaluation metric is the *final success rate*, defined as the agent’s ability to successfully synthesize the target item by strictly adhering to the unseen rules.

4. Pre-Training for Multi-Turn Long-Horizon Planning

Current large language model agents remain weak at multi-turn long-horizon planning. We investigate this problem by progressively relaxing three assumptions about pre-training data. (1) **World-model internalization.** Given correct expert trajectories, does explicitly modeling intermediate state transitions improve planning beyond action-sequence

imitation? (2) **Long-tail compositionality**. When long-horizon trajectories are limited, can models compose abundant atomic skills to generalize to more complex planning tasks? (3) **Planning patterns and knowledge quality**. When expert trajectories are mixed with redundant, suboptimal, or erroneous solutions, how does this heterogeneity affect the acquisition and scaling of planning patterns and planning knowledge? Accordingly, this section studies **Planning with Internalized Skill World Model**, **Planning Internalized Skill Compositional Generalization**, and **Heterogeneous Planning Patterns and Planning Knowledge**.

4.1. Planning with Internalized Skill World Model

World Modeling. Fundamentally, a world model represents the underlying dynamics of an environment, serving as an internal simulator of how the world evolves. At its core, the world model is defined by a state transition function:

$$\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$$

where $\mathcal{S}$ denotes the state space and $\mathcal{A}$ denotes the action space. Given a current state $s_t \in \mathcal{S}$ and an action $a_t \in \mathcal{A}$, the transition function predicts the resulting next state $s_{t+1} = \mathcal{T}(s_t, a_t)$. This predictive capability allows an agent to simulate and evaluate the future consequences of its decisions purely through internal reasoning, without interacting with the external environment.

Skill World Modeling. We model planning as depth-first search over an internal simulated state space $\mathcal{S}$. Let $\mathcal{K}$ denote the set of atomic skills, where each skill $k \in \mathcal{K}$ is defined as a state transition function $f_k : \mathcal{S} \rightarrow \mathcal{S}$. We define a planning step t as one internal DFS node expansion. At each step, the model selects an ordered skill sequence $\mathbf{k}_t = (k_{t,1}, \dots, k_{t,m_t}) \in \mathcal{K}^*$ and applies the corresponding transition operators sequentially on the internal state:

$$s_{t+1} = f_{k_{t,m_t}} \circ \dots \circ f_{k_{t,1}}(s_t),$$

which represents the cumulative effect of executing the selected skills within the internal world model.

The resulting transitions define a search tree over simulated states, and the model performs depth-first search by recursively expanding one branch, backtracking upon failure, and exploring alternative skill sequences until a valid plan is found. All transitions and search operations are conducted within the chain-of-thought internal world model rather than through external environment interaction.

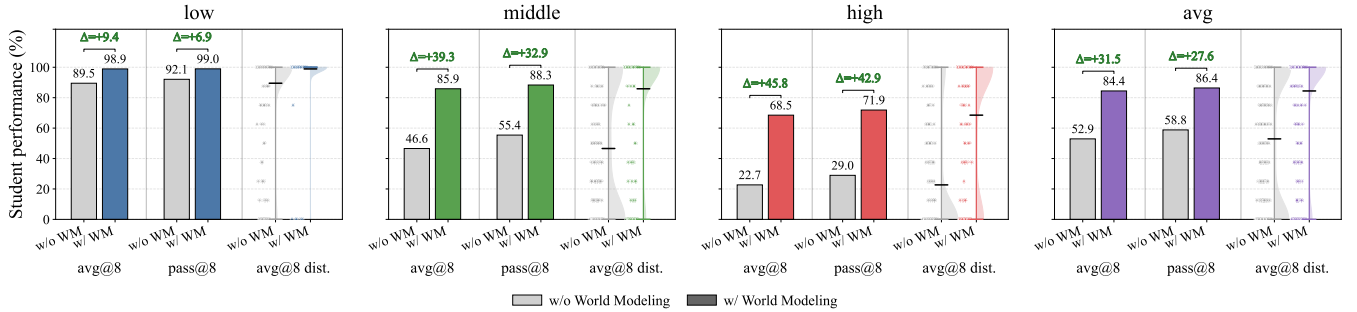


Figure 3: Final performance comparison between students with and without world modeling across three difficulty levels. For 3 domain and 3 difficulty level, each test subset contains 160 instances, resulting in 1,440 test instances for each checkpoint. During inference, we set the sampling temperature to 0.4 and independently sample 8 trajectories for each instance, with a maximum horizon of 20 steps per trajectory. Therefore, evaluating one checkpoint requires a total of 11,520 inference runs. We report avg@8 as the average success rate across the 8 sampled trajectories, and pass@8 as the proportion of instances for which at least one of the 8 trajectories successfully completes the task. Unless otherwise specified, we use the same evaluation protocol in all subsequent experiments.

Task Setup. (1) **Model configuration**. Since the pre-training corpora of existing models are proprietary and inaccessible, we instead construct a controlled experimental setting by randomly initializing a model under the Qwen2.5 architecture. To ensure consistent tokenization in the synthetic pre-training corpus, we train a byte-level BPE tokenizer from scratch. The full model configuration is detailed in Appendix A.1. (2) **Agent planning task configuration**. The corpus covers 3 domains. Each domain contains 5 levels, with 40 item categories per level and 20 instances per category. These item categories have composition rules, meaning an item can be made from other items using AND

and OR logic. The task requires the model to create a target item from basic materials, while extra noise items are added to the mix. You can find the details in Appendix A.2. (3) **Agent planning corpus configuration.** We compare the performance of direct answering and world modeling planning. For direct answering, the model predicts the action for each step directly. For world modeling planning, the model must plan before acting. Specifically, it builds a chain of thought using post order traversal, state transition rules for item composition, and grounding from categories to exact instances. We train 1.2B corpus for the student model. You can find the exact corpus setup in Appendix A.3. And training details you can find in Appendix A.4.

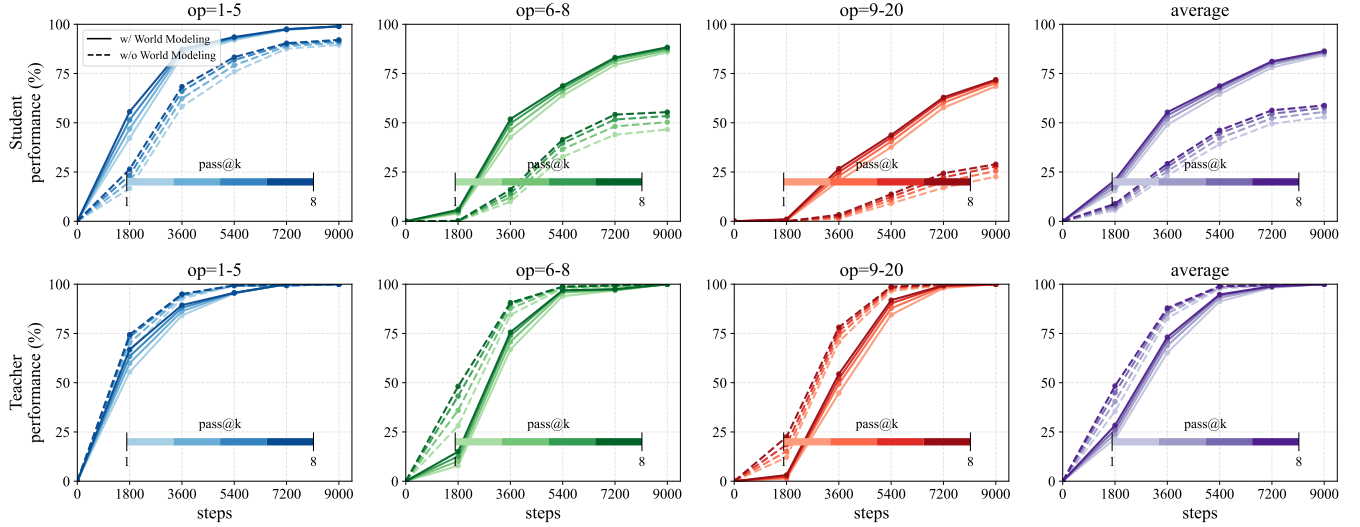


Figure 4: Performance scaling with pre-training corpus size for agent planning, comparing students with and without world modeling across three difficulty levels. The teacher is demonstrating the performance of the post-training set with the same distribution.

Results. (1) **Internalizing a world model leads to stronger generalization ability.** As shown in Figure 3, the student model equipped with a world model (w/ WM) consistently outperforms the direct answering baseline (w/o WM) across all difficulty levels. Specifically, the model shows improvements in the low difficulty setting of +9.4% (from 89.5% to 98.9%) and +6.9% (from 92.1% to 99.0%) for avg@8 and pass@8, respectively. Similar enhancements are observed in the middle ($\Delta = +39.3\%$ for avg@8, $\Delta = +32.9\%$ for pass@8) and high ($\Delta = +45.8\%$ for avg@8, $\Delta = +42.9\%$ for pass@8) difficulty levels, proving that planning with an internal world model improves the agent’s ability to generalize to new and complex planning tasks. (2) **Direct answering converges faster, but suffers from a lower upper bound.** As shown in Figure 4, analyzing the performance scaling curves over training steps reveals a distinct trade-off. In terms of learning speed, direct prediction (w/o WM, dashed lines) exhibits a faster initial trajectory. This is clearly evident in the teacher performance curves, where the dashed lines rise more rapidly and reach high performance at earlier stages (e.g., between 1800 and 5400 steps) compared to the solid lines (w/ WM). However, direct answering is limited by a strictly lower performance ceiling. As demonstrated by the student performance curves across all operation ranges (op=1-5, op=6-8, op=9-20) and their average, the solid lines (w/ WM) eventually surpass the dashed lines and stabilize at a significantly higher final accuracy. This indicates that while explicitly modeling state transitions via chain-of-thought slows down initial convergence, it finally unlocks a much higher upper bound for multi-turn long-horizon planning.

Takeaway

Internalized world models enhance multi-turn planning generalization. Explicitly modeling state transitions improves performance. While direct answering converges faster initially, planning with a world model ultimately surpasses it to achieve a higher final accuracy.

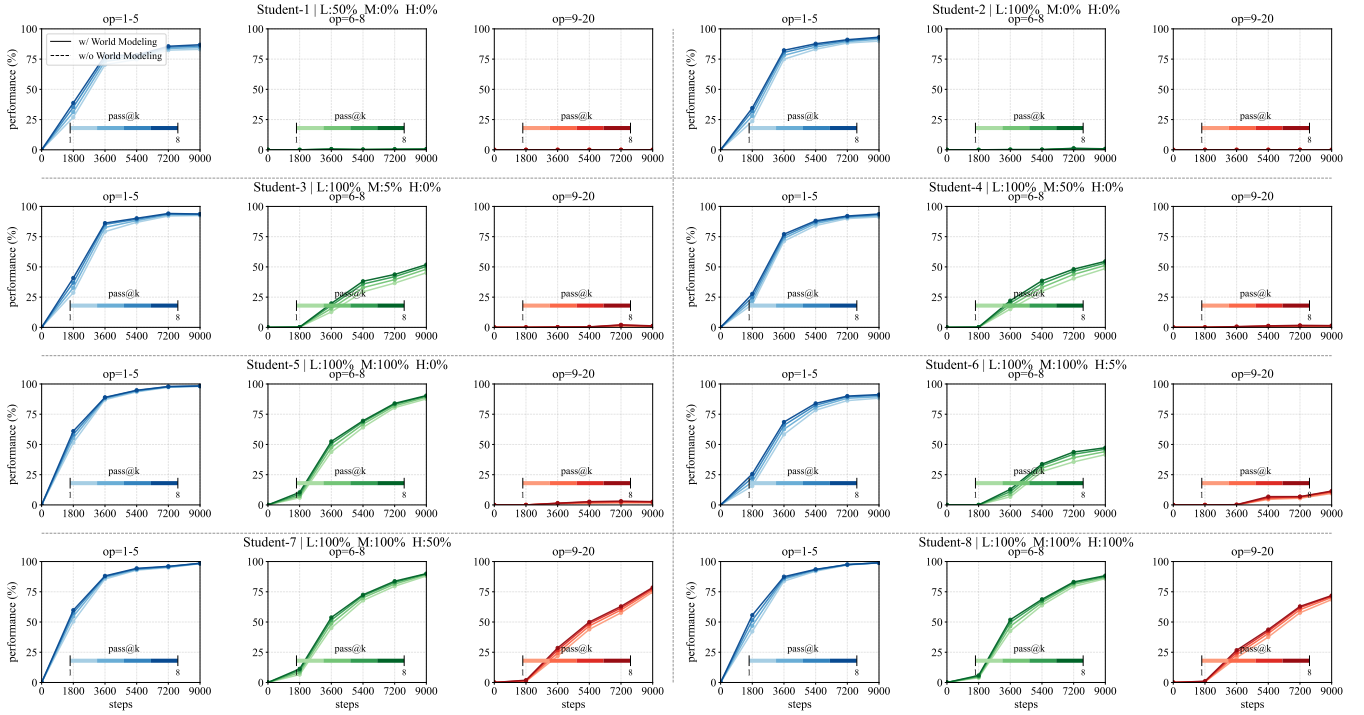


Figure 5: Performance scaling across different pretraining distributions.

4.2. Impact of Planning Internalized Skill Compositional Generalization

Skill Compositional Generalization. Compositional generalization refers to the ability of an agent to combine known **atomic skills** into new **compositional skills** for solving unseen complex tasks. In this work, such compositional ability is acquired through pre-training and encoded in the model parameters, rather than elicited through test-time demonstrations, prompt engineering, or in-context learning. In our environment, an atomic skill is the ability to complete a single valid synthesis step. We represent such a step as a basic function $f(x)$ that produces a new item from a set of input materials. Suppose the model learns two recipes, f and g , from the training corpus. Compositional generalization occurs when, without having observed their complete composition during training, the model can correctly execute the novel nested pathway

$$h(x) = g(f(x))$$

at test time. In our hierarchical graph tasks, this requires the model to synthesize a target node at a higher level by chaining independently learned lower-level recipes. This setting therefore evaluates whether pre-training enables the model to internalize reusable synthesis rules and systematically compose them, rather than merely memorizing complete trajectories or relying on test-time prompting.

Task Setup. (1) **Difficulty Definition.** We divide the tasks into low, middle, and high levels based on the number of operation steps. Specifically, the low level requires 1 to 5 steps, the middle level requires 6 to 8 steps, and the high level requires 9 or more steps. (2) **Data allocation.** We organize the training data into three difficulty intervals. The higher level interval includes all data from the lower level intervals. The lower level intervals do not contain complete tasks from the higher level, but only partial task components. Within each interval, we set different data ratios including 0%, 5%, 50%, and 100%. (3) **Experiment and evaluation.** All experiments use a unified number of training steps for 9000 steps. We then evaluate the models across the three difficulty levels to test the compositional generalization ability of their internalized skills.

Results. As shown in Figure 5 and Table 1. (1) **Atomic skills struggle to compose automatically.** When the model is pretrained exclusively on short tasks (Region L at 100%), it achieves strong performance on Low-level tasks (PASS@8 of 93.12%), but accuracy on longer horizons drops to near zero (0.83% for Middle and 0.00% for High levels). For multi-turn agents, simply knowing individual actions is insufficient; they fail to connect atomic skills into

Table 1: Pretrain distribution table with two global blocks (top: avg@8, bottom: pass@8). Ratio is split into L(Low)/M(Middle)/H(High) columns. Each difficulty includes domain columns and a mean column. Two extra columns report difficulty gaps: Gap(H-L) and Gap(M-L). Domain abbreviations: FA = Fantasy Alchemy; LF = Livestock Farming; EA = Electronic Assembly.

Region	Ratio			Low				Middle				High				Gap	
	L	M	H	FA	LF	EA	Mean	FA	LF	EA	Mean	FA	LF	EA	Mean	H-L	M-L
AVG@8																	
L	50%	0%	0%	90.62	74.84	83.75	83.07	0.23	0.00	0.55	0.26	0.00	0.00	0.00	0.00	-83.07	-82.81
	100%	0%	0%	95.55	80.70	93.28	89.84	0.62	0.16	0.39	0.39	0.00	0.00	0.00	0.00	-89.84	-89.45
L+M	100%	5%	0%	96.88	84.53	96.09	92.50	56.95	37.19	41.25	45.13	0.70	0.62	1.02	0.78	-91.72	-47.37
	100%	50%	0%	94.14	88.83	90.86	91.28	59.69	38.52	47.73	48.65	0.55	0.00	1.95	0.83	-90.44	-42.63
	100%	100%	0%	99.77	97.42	96.72	97.97	89.38	87.50	86.09	87.66	0.70	1.72	2.81	1.74	-96.22	-10.31
L+M+H	100%	100%	5%	93.52	80.62	90.62	88.26	58.98	31.02	34.77	41.59	13.52	3.20	11.72	9.48	-78.78	-46.67
	100%	100%	50%	99.92	96.56	98.12	98.20	88.12	85.55	90.55	88.07	78.05	72.50	73.98	74.84	-23.36	-10.13
	100%	100%	100%	100.00	99.22	97.50	98.91	91.80	82.19	83.59	85.86	69.61	65.55	70.39	68.52	-30.39	-13.05
PASS@8																	
L	50%	0%	0%	92.50	81.25	86.88	86.88	1.25	0.00	0.62	0.62	0.00	0.00	0.00	0.00	-86.88	-86.25
	100%	0%	0%	97.50	86.25	95.62	93.12	0.62	0.62	1.25	0.83	0.00	0.00	0.00	0.00	-93.12	-92.29
L+M	100%	5%	0%	98.12	86.25	96.88	93.75	61.88	43.12	50.62	51.88	1.25	0.62	1.88	1.25	-92.50	-41.88
	100%	50%	0%	95.00	93.12	93.12	93.75	63.12	45.00	55.62	54.58	0.62	0.00	3.75	1.46	-92.29	-39.17
	100%	100%	0%	100.00	98.12	96.88	98.33	91.25	90.00	90.00	90.42	1.25	2.50	4.38	2.71	-95.62	-7.92
L+M+H	100%	100%	5%	95.62	85.00	93.12	91.25	63.12	35.00	43.75	47.29	16.88	3.75	13.75	11.46	-79.79	-43.96
	100%	100%	50%	100.00	97.50	98.12	98.54	89.38	88.75	91.88	90.00	81.88	76.25	77.50	78.54	-20.00	-8.54
	100%	100%	100%	100.00	99.38	97.50	98.96	93.75	85.00	86.25	88.33	71.88	71.25	72.50	71.88	-27.08	-10.62

long trajectories without explicitly observing extended state transitions during training. (2) **Minimal exposure to long-horizon trajectories activate planning generalization.** Adding just a small fraction of long horizon data causes a sudden jump in performance on harder levels. For instance, introducing merely 5% of Middle-level data boosts mean Middle PASS@8 from 0.83% to 51.88%, while 5% of High-level data increases High PASS@8 from 0.00% to 11.46%.

Takeaway

Models struggle with automatic compositional generalization, but minimal long-horizon data activates long-range planning. Learning atomic skills exclusively from short tasks is insufficient for solving multi-step problems. However, introducing just a small fraction of long-trajectory data teaches the agent how to connect these skills sequentially, successfully bridging the performance gap on harder tasks.

4.3. Heterogeneous Planning Patterns and Planning Knowledge

Planning Pattern. We define the planning pattern as the structural and algorithmic skeleton of the CoT. It dictates the logical sequence of operations required to solve the synthesis problem, independent of the specific entities involved. As demonstrated in Table 2, the planning pattern manifests as the systematic procedure of identifying the target category, retrieving the prerequisite rule, mapping abstract categories to concrete inventory items, and formulating the final action. This invariant template relies on standardized textual scaffolds, employing consistent phrasing to explicitly declare the target objective, transition to prerequisite analysis, and finally enumerate the selected materials.

Planning Knowledge. In contrast, we define planning knowledge as the task specific semantic entities, categorical relationships, and topological constraints required to ground the abstract pattern into a valid mathematical or logical solution. It represents the actual factual content and graph structure injected into the reasoning skeleton. As illustrated in Table 2, the planning knowledge encompasses the explicit synthesis rules defining exactly which materials combine to create a new item. This refers to the factual recipes within the skill graph, such as the specific knowledge that

combining Emberr Moss and Tinpebble synthesizes a Moon Infusion.

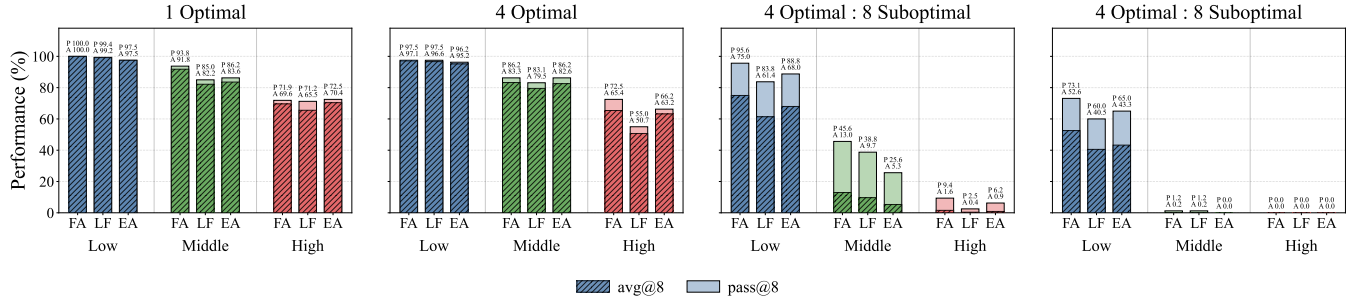


Figure 6: Effect of pretraining data composition on downstream performance. The figure compares avg@8 and pass@8 across three task domains and three difficulty levels under different pretraining configurations. FA, LF, and EA denote Fantasy Alchemy, Livestock Farming, and Electronic Assembly. Dark and light bars correspond to avg@8 and pass@8, respectively.

Task Setup. (1) **Planning Mode.** We primarily establish two planning modes. The first thinking mode features a template and an optimal single step decision. The second thinking mode combines a template and a suboptimal decision. Furthermore, each template corresponds directly to whether it represents an optimal decision. (2) **Planning Mode Distribution.** We configure four mode distributions. These distributions include one optimal mode, four optimal modes, a four to four ratio between optimal and suboptimal modes, and a four to eight ratio between optimal and suboptimal modes. In addition, all data utilize the same metadata except for different rendering, while the data scale and training steps remain consistent.

Results. As shown in Figure 6. (1) **Multiple optimal patterns maintain similar generalization.** Increasing the diversity of optimal reasoning templates from 1 Optimal to 4 Optimal does not significantly degrade performance. The model maintains similar success rates across all three domains (FA, LF, EA) and smoothly generalizes to Middle and High difficulty levels. This indicates that agents can effectively internalize and utilize multiple high-quality structural skeletons without detrimental interference. (2) **Suboptimal trajectories cause catastrophic collapse due to compounding errors.** The introduction of suboptimal decisions (4 Optimal : 8 Suboptimal) severely undermines the model’s planning capabilities. While the model retains partial capability on Low-difficulty tasks, its performance plummets to near-zero on Middle and High-difficulty tasks. In sequential decision-making, a suboptimal step early in the trajectory does not merely cause a localized failure; rather, the error accumulates and amplifies step-by-step. While agents might tolerate minor deviations in short-horizon tasks, these cascading errors in extended plans steer the agent irrecoverably off course, preventing it from completing the objective within the required step limits.

Takeaway

Imperfect pre-training data severely degrades long-horizon planning. In real-world settings, pre-training data naturally contains non-optimal trajectories instead of perfect expert demonstrations. These sub-optimal paths introduce reasoning errors that accumulate heavily over long horizons. When trained on this mixed data, the agent fails to complete long tasks. Trying to overcome these learned flaws by simply increasing sampling is ineffective, as it fails to improve the success rate in long-horizon tasks. Furthermore, in multi-turn scenarios, relying on massive sampling causes unacceptable computational costs, making it impractical for real-world deployments.

5. Single Expert On-Policy Agentic Distillation

5.1. Mutual Information for Planning Pattern and Planning Knowledge

Mutual Information. We use mutual information to measure the relevance between the input task (T) and the two distinct outputs: planning pattern (P) and planning knowledge (K). To reflect their different properties, we define their mutual information separately as:

$$I(T; P) = H(P) - H(P|T), \quad I(T; K) = H(K) - H(K|T) \quad (7)$$

Table 2: Planning pattern and planning knowledge for the first fantasy alchemy test sample.

Part	JSON Content
Input	<pre> <question> {"task": "Synthesize starquench vial", "target_category": "Star Elixir", "inventory": ["Crystal Speck", "Dragon Scute", "Faerie Dusting", "Glow Frond", "Lunar Sprig", "Mithril Lump", "Obsidian Chip", "Sapphire Grit", "Sylph Filament"], "inventory_category_description": ["Crystal Speck belongs to category Crystalgrain.", "Dragon Scute belongs to category Dragon Scale.", "Faerie Dusting belongs to category Faerie Pollen.", "Glow Frond belongs to category Glowfern.", "Lunar Sprig belongs to category Moonleaf.", "Mithril Lump belongs to category Mithril Nugget.", "Obsidian Chip belongs to category Obsidian Shard.", "Sapphire Grit belongs to category Sapphire Sand.", "Sylph Filament belongs to category Sylph Thread."], "history": []} </question> </pre>
Output	<pre> <solution> Goal: synthesize Star Elixir. Category synthesis rule: combining Ruby Matrix, Unicorn Matrix can produce Star Elixir. Ruby Matrix is currently not represented in inventory. Unicorn Matrix is currently not represented in inventory. Status: need to acquire Ruby Matrix, Unicorn Matrix first. Goal: synthesize Ruby Matrix. Category synthesis rule: combining Nymph Gel, Quartz Solution can produce Ruby Matrix. Nymph Gel is currently not represented in inventory. Quartz Solution is currently not represented in inventory. Status: need to acquire Nymph Gel, Quartz Solution first. Goal: synthesize Nymph Gel. Category synthesis rule: combining Mithril Nugget, Sylph Thread can produce Nymph Gel. Mithril Nugget is represented by items like Mithril Lump. Sylph Thread is represented by items like Sylph Filament. Action: use Mithril Lump (Mithril Nugget), Sylph Filament (Sylph Thread) to fulfill this category requirement. </solution> <answer>{"type": "process", "materials": ["Mithril Lump", "Sylph Filament"]} </answer> </pre>
Skill Graph	<pre> graph LR CS["Crystal Speck (Crystalgrain)"] -- cyan --> SL["silica liquor (Quartz Solution)"] GF["Glow Frond (Glowfern)"] -- cyan --> SL ML["Mithril Lump (Mithril Nugget)"] -- blue --> SJ["spring jelly (Nymph Gel)"] SF["Sylph Filament (Sylph Thread)"] -- blue --> SJ DS["Dragon Scute (Dragon Scale)"] -- orange --> CM["coalroot mash (Ember Paste)"] FD["Faerie Dusting (Faerie Pollen)"] -- orange --> CM LS["Lunar Sprig (Moonleaf)"] -- purple --> UN["unicornlace node (Unicorn Matrix)"] CM -- purple --> UN SL -- pink --> RS["rubylace shard (Ruby Matrix)"] SJ -- pink --> RS RS -- red --> SV["starquench vial (Star Elixir)"] UN -- red --> SV </pre>
Suboptimal Output	<pre> <solution>Direct selection from current inventory.</solution> <answer>{"type": "process", "materials": ["Lunar Sprig", "Obsidian Chip"]} </answer> </pre>

where $I(T; P)$ and $I(T; K)$ quantify the reduction in uncertainty about P and K , respectively, when the task T is given. By decoupling them, the relationship between task relevance and cross-sample generality becomes evident. The planning pattern P captures shared structural information with high cross-sample generality, making it less dependent on specific task details and resulting in a relatively lower $I(T; P)$. In contrast, the planning knowledge K represents highly task-specific details, meaning it is strongly constrained by the input task, thereby yielding a higher $I(T; K)$.

To illustrate, general planning patterns P , such as reflection or backtracking, often span across diverse domains. Conversely, the planning knowledge K (e.g., specific skills and procedural knowledge) is highly idiosyncratic and tightly bound to the current state of a specific sample. While different tasks certainly necessitate different reasoning templates, the input task T imposes far stricter constraints on the specific factual details required than on the broad strategy used. This relative disparity implies that knowing T reduces significantly more uncertainty about K than about P , establishing that $I(T; P) < I(T; K)$.

5.2. Impact of Planning Pattern

5.2.1. Accuracy Analysis

Training Setup. (1) **Base models.** As shown in Section 4.3, we use three pre-trained base models: 4 Opt, 4 Opt:4 Sub, and 4 Opt:8 Sub. (2) **RL data.** We use the post-training data from Table 11. To test cross-task generalization, we entirely use the Fantasy Alchemy subset as the post-training data. To test cross-difficulty generalization, we use data from low, middle, and high difficulty levels. We downsample 3000 data items during post-training, but training process may not use all of them. (3) **RL algorithms.** We use multi-turn GRPO and OPD algorithms. GRPO uses the final outcome reward. OPD uses Top-k OPD, where K is 100.

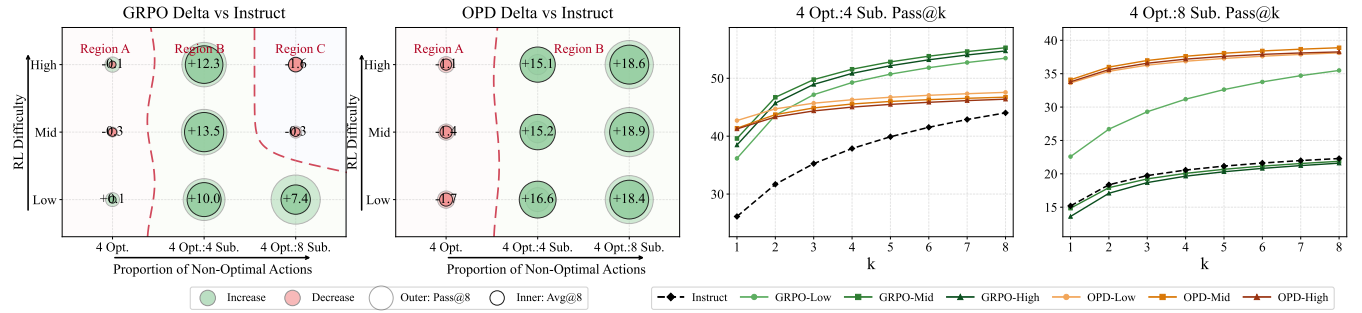


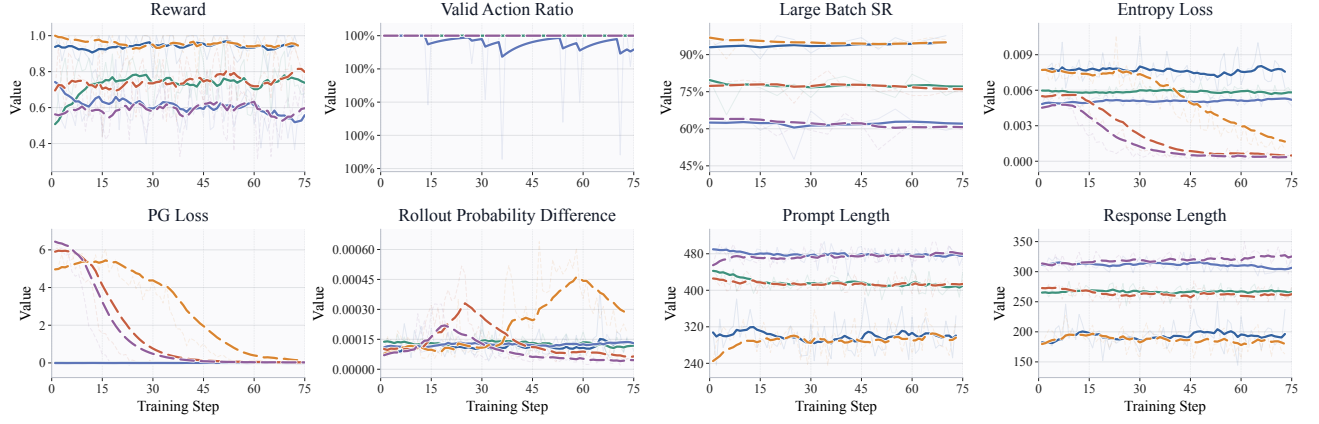
Figure 7: Applicability boundaries and post-training capability upper bounds of RL training. The first two panels show the applicability boundaries of GRPO and OPD across RL difficulty and non-optimal action ratios. The last two panels compare the post-training capability upper bounds of different models.

Results. As shown in Figure 7, Table 3, and Figure 8. (1) **GRPO and OPD have applicable boundaries based on RL difficulty and non optimal actions.** There are three specific regions. **Region A is the knowledge injection region. Knowledge tokens resist RL and must be learned through SFT.** For GRPO and OPD, even if the teacher performance is 100%, high mutual information planning knowledge tokens are difficult to improve via RL and hard to generalize across samples. These tokens need to be injected via SFT. **Region B is the suitable region for RL. Low mutual information patterns generalize well through RL.** Low mutual information planning pattern tokens are easy to stimulate via RL and can generalize across samples, difficulties, and datasets. This is essentially reasoning pattern selection for SFT (fundamentally, selecting low mutual information tokens). **Region C is the limitation region for long horizon tasks. Long horizon tasks confuse outcome rewards and require fine grained reward methods.** In long horizon tasks, when both non optimal actions and RL difficulty increase, correct and incorrect actions are mixed in very long trajectories. ORM based GRPO will assign rewards incorrectly, failing to optimize long horizon agent tasks. In contrast, fine grained reward methods like OPD provide a promising approach. (2) **Pass@k is not always a good metric for the upper limit of model capability in multiple turn long horizon agent scenarios.** In the 4 Opt.:4 Sub. region, OPD has a higher pass@1, but as k increases, GRPO has a higher performance limit. However, for the 4 Opt.:8 Sub. region, short term GRPO is effective, but GRPO shows almost no improvement for long horizon tasks. Meanwhile, OPD shows better pass@1 and pass@8. This indicates that GRPO generates the best planning pattern based on the environment, showing stronger potential. However, OPD suppresses diversity and directly follows the planning pattern

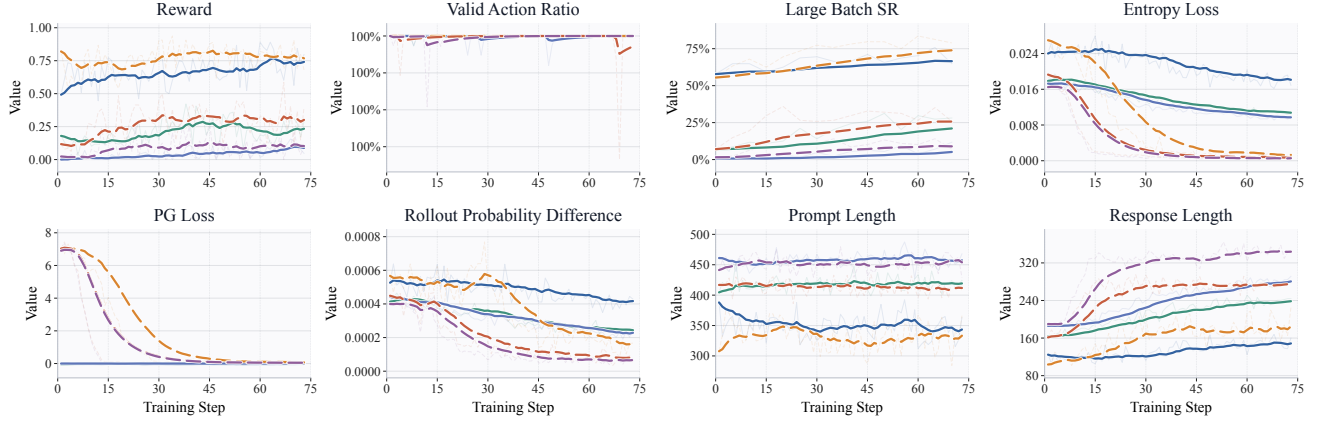
Table 3: Pattern-base comparison table. Rows are grouped by the pattern mixture: 4 Opt., 4 Opt.:4 Sub., and 4 Opt.:8 Sub. Each group contains the instruct checkpoint and OPD/GRPO models trained on low, middle, and high difficulty. Two global blocks are reported (top: Avg@8, bottom: Pass@8). The last column reports the mean over low/middle/high evaluation difficulty means and its triangle delta vs. the instruct baseline in the same group. Domain abbreviations: FA = Fantasy Alchemy; LF = Livestock Farming; EA = Electronic Assembly.

Pattern Mix	Model	Low				Middle				High				Avg	
		FA	LF	EA	Mean	FA	LF	EA	Mean	FA	LF	EA	Mean	L/M/H Mean (Δ)	
AVG@8															
4 Opt.	Instruct	97.11	96.64	95.16	96.30	83.28	79.53	82.58	81.80	65.39	50.70	63.20	59.77	79.29	$\Delta+0.00$
	Low OPD	97.27	94.77	95.55	95.86	81.95	76.64	78.52	79.04	63.12	48.98	61.88	57.99	77.63	$\Delta-1.66$
	Low GRPO	97.03	96.88	95.16	96.35	83.28	79.14	82.89	81.77	65.47	50.86	63.75	60.03	79.38	$\Delta+0.10$
	Mid OPD	97.81	95.47	94.92	96.07	81.33	78.83	78.67	79.61	64.61	48.44	61.09	58.05	77.91	$\Delta-1.38$
	Mid GRPO	96.95	96.64	95.08	96.22	82.34	78.20	82.73	81.09	64.45	51.09	63.52	59.69	79.00	$\Delta-0.29$
	High OPD	96.88	95.55	95.31	95.91	81.17	78.83	82.66	80.89	61.88	50.08	61.48	57.81	78.20	$\Delta-1.09$
	High GRPO	96.80	96.48	95.62	96.30	82.11	77.66	82.66	80.81	65.47	52.50	63.52	60.49	79.20	$\Delta-0.09$
4 Opt.:4 Sub.	Instruct	75.00	61.41	67.97	68.12	12.97	9.69	5.31	9.32	1.56	0.39	0.86	0.94	26.13	$\Delta+0.00$
	Low OPD	93.52	71.02	80.08	81.54	45.78	32.97	25.39	34.71	15.47	6.02	14.06	11.85	42.70	$\Delta+16.57$
	Low GRPO	85.78	69.84	75.39	77.01	34.14	26.80	15.47	25.47	9.30	2.42	6.33	6.02	36.16	$\Delta+10.03$
	Mid OPD	91.25	71.56	79.77	80.86	42.19	30.86	26.48	33.18	12.89	6.41	10.94	10.08	41.37	$\Delta+15.24$
	Mid GRPO	87.73	70.00	77.58	78.44	42.42	31.25	20.62	31.43	13.91	4.38	8.52	8.93	39.60	$\Delta+13.47$
	High OPD	92.03	70.78	79.92	80.91	40.39	33.12	24.61	32.71	13.28	4.53	12.58	10.13	41.25	$\Delta+15.12$
	High GRPO	86.95	69.30	77.66	77.97	37.66	29.38	19.77	28.93	14.22	2.97	8.36	8.52	38.47	$\Delta+12.34$
4 Opt.:8 Sub.	Instruct	52.58	40.55	43.28	45.47	0.16	0.16	0.00	0.10	0.00	0.00	0.00	0.00	15.19	$\Delta+0.00$
	Low OPD	85.86	62.58	72.66	73.70	29.22	22.34	16.02	22.53	6.88	3.52	3.44	4.61	33.61	$\Delta+18.42$
	Low GRPO	71.48	55.08	60.55	62.37	6.64	6.80	1.88	5.10	0.23	0.39	0.16	0.26	22.58	$\Delta+7.39$
	Mid OPD	86.72	64.06	71.95	74.24	33.44	20.47	14.84	22.92	8.75	3.91	2.66	5.10	34.09	$\Delta+18.90$
	Mid GRPO	52.11	38.98	42.42	44.51	0.08	0.08	0.00	0.05	0.00	0.00	0.00	0.00	14.85	$\Delta-0.34$
	High OPD	85.00	62.89	71.80	73.23	31.64	20.78	16.09	22.84	7.81	3.67	4.38	5.29	33.78	$\Delta+18.59$
High GRPO	45.55	37.81	38.91	40.76	0.08	0.16	0.00	0.08	0.00	0.00	0.00	0.00	13.61	$\Delta-1.58$	
PASS@8															
4 Opt.	Instruct	97.50	97.50	96.25	97.08	86.25	83.12	86.25	85.21	72.50	55.00	66.25	64.58	82.29	$\Delta+0.00$
	Low OPD	98.12	96.88	95.62	96.88	85.00	80.00	82.50	82.50	68.75	54.38	65.00	62.71	80.69	$\Delta-1.60$
	Low GRPO	97.50	97.50	96.25	97.08	88.12	83.12	88.12	86.46	72.50	56.25	67.50	65.42	82.99	$\Delta+0.69$
	Mid OPD	98.12	96.25	95.62	96.67	85.62	79.38	83.12	82.71	68.75	55.00	66.88	63.54	80.97	$\Delta-1.32$
	Mid GRPO	97.50	97.50	96.25	97.08	86.25	83.12	86.25	85.21	68.12	57.50	66.88	64.17	82.15	$\Delta-0.14$
	High OPD	97.50	96.88	95.62	96.67	85.00	80.00	86.88	83.96	68.12	54.38	64.38	62.29	80.97	$\Delta-1.32$
	High GRPO	97.50	97.50	95.62	96.88	86.88	81.88	89.38	86.04	71.88	58.75	69.38	66.67	83.19	$\Delta+0.90$
4 Opt.:4 Sub.	Instruct	95.62	83.75	88.75	89.38	45.62	38.75	25.62	36.67	9.38	2.50	6.25	6.04	44.03	$\Delta+0.00$
	Low OPD	95.62	78.12	84.38	86.04	51.25	40.00	31.25	40.83	20.00	8.75	18.75	15.83	47.57	$\Delta+3.54$
	Low GRPO	96.88	86.25	92.50	91.88	63.12	47.50	40.00	50.21	25.00	9.38	20.62	18.33	53.47	$\Delta+9.44$
	Mid OPD	92.50	77.50	85.00	85.00	51.25	36.88	34.38	40.83	17.50	10.00	15.62	14.38	46.74	$\Delta+2.71$
	Mid GRPO	96.88	86.25	91.88	91.67	66.25	46.88	45.62	52.92	27.50	13.12	23.12	21.25	55.28	$\Delta+11.25$
	High OPD	93.75	75.62	85.00	84.79	47.50	39.38	34.38	40.42	17.50	7.50	16.88	13.96	46.39	$\Delta+2.36$
High GRPO	96.88	86.88	90.62	91.46	61.88	46.88	43.75	50.83	32.50	11.88	21.25	21.88	54.72	$\Delta+10.69$	
4 Opt.:8 Sub.	Instruct	73.12	60.00	65.00	66.04	1.25	1.25	0.00	0.83	0.00	0.00	0.00	0.00	22.29	$\Delta+0.00$
	Low OPD	87.50	68.12	78.12	77.92	35.00	27.50	21.88	28.12	12.50	5.00	7.50	8.33	38.12	$\Delta+15.83$
	Low GRPO	90.62	78.12	80.62	83.12	28.12	25.00	11.25	21.46	1.88	2.50	1.25	1.88	35.49	$\Delta+13.19$
	Mid OPD	88.75	71.25	78.75	79.58	41.25	26.25	20.62	29.38	12.50	5.62	5.00	7.71	38.89	$\Delta+16.60$
	Mid GRPO	71.25	58.12	66.25	65.21	0.62	0.62	0.00	0.42	0.00	0.00	0.00	0.00	21.88	$\Delta-0.42$
	High OPD	89.38	70.00	76.25	78.54	36.25	27.50	21.88	28.54	11.25	5.62	6.25	7.71	38.26	$\Delta+15.97$
	High GRPO	70.62	59.38	62.50	64.17	0.62	1.25	0.00	0.62	0.00	0.00	0.00	0.00	21.60	$\Delta-0.69$

4 Opt.



4 Opt.:4 Sub.



4 Opt.:8 Sub.

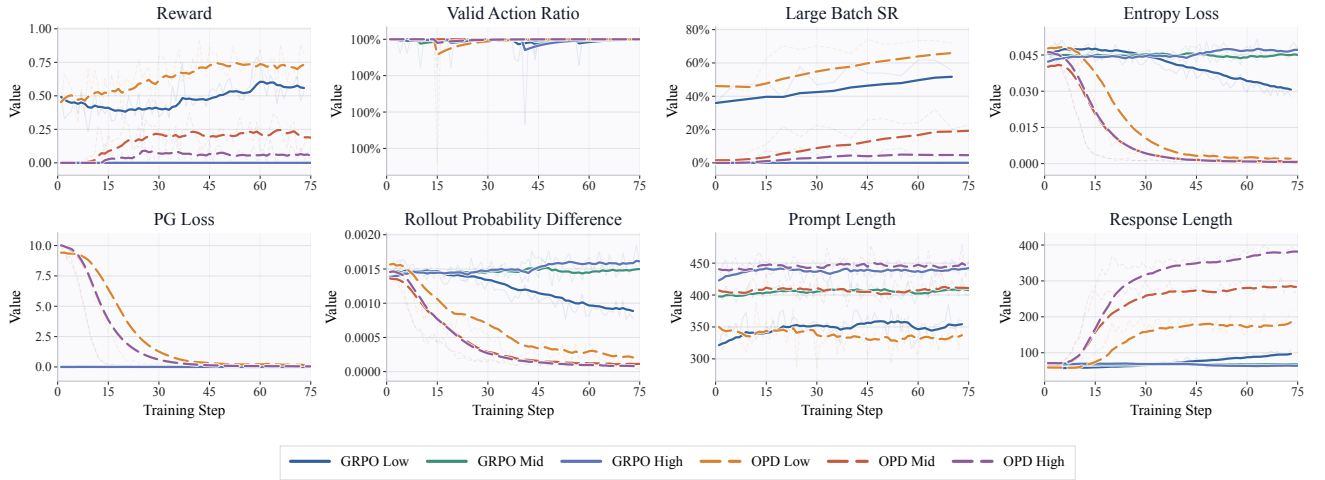


Figure 8: Training dynamics across optimization settings. Comparison of GRPO and OPD under 4 Opt., 4 Opt.:4 Sub., and 4 Opt.:8 Sub. settings.

of the teacher. Therefore, it is clear that the correctness of the teacher planning pattern is extremely important. Please note that we do not consider pass@k to represent the upper limit of model capability in long horizon scenarios. In later sections, we will control the pretraining data to directly control this upper limit. (3) **Chain of thought length changes after RL depend on the reasoning pattern.** After training, the output length of the model will change, which is related to the data distribution during pretraining. If the better planning pattern is longer, the trained output length will increase. If the better planning pattern is shorter, the trained output length will decrease. Experiments for the shorter patterns will be provided in the appendix.

Takeaway

Knowledge requires SFT. High mutual information planning knowledge resists RL optimization and must be injected during supervised fine tuning.

Patterns generalize via RL. Low mutual information reasoning patterns are easily stimulated by RL and generalize well across different samples and difficulties.

Long horizon tasks need fine grained credit assignment. Outcome based rewards struggle when correct and incorrect actions mix in long trajectories, making fine grained methods necessary.

Pass@k fails to show capability limits. In multiple turn long horizon scenarios, pass@k is not a reliable metric for the upper limit of model capability.

Output length depends on patterns. The chain of thought length after RL can increase or decrease based on the optimal reasoning pattern found in the pretraining data.

5.2.2. Gradient Direction Analysis

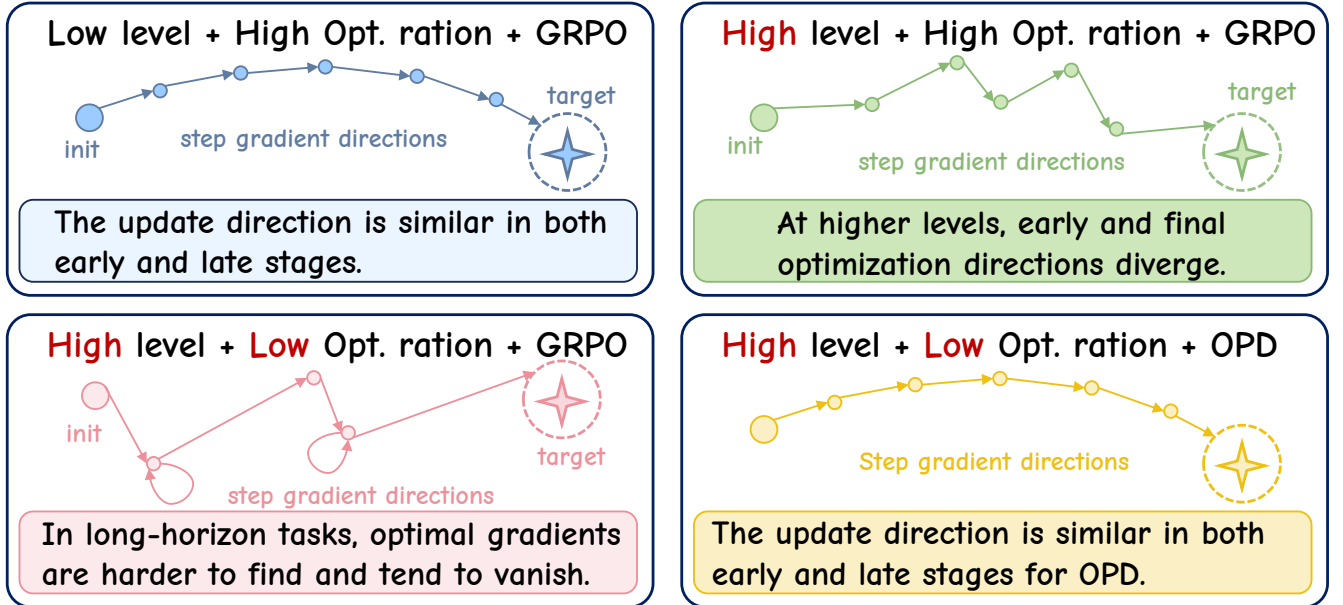


Figure 9: Comparison of step gradient directions between GRPO and OPD under varying optimization conditions.

Top-k Subspace for SVD. Beyond measuring task performance, we analyze whether the model already follows the final parameter-update direction in the early stages of training. Specifically, we compare the dominant update directions of early and intermediate checkpoints with those of the final checkpoint. Let $W_\ell^{(t)}$ be the weight matrix of layer ℓ at checkpoint t , and let b be the base checkpoint. We first define the update matrix

$$\Delta W_\ell^{(t)} = W_\ell^{(t)} - W_\ell^{(b)}.$$

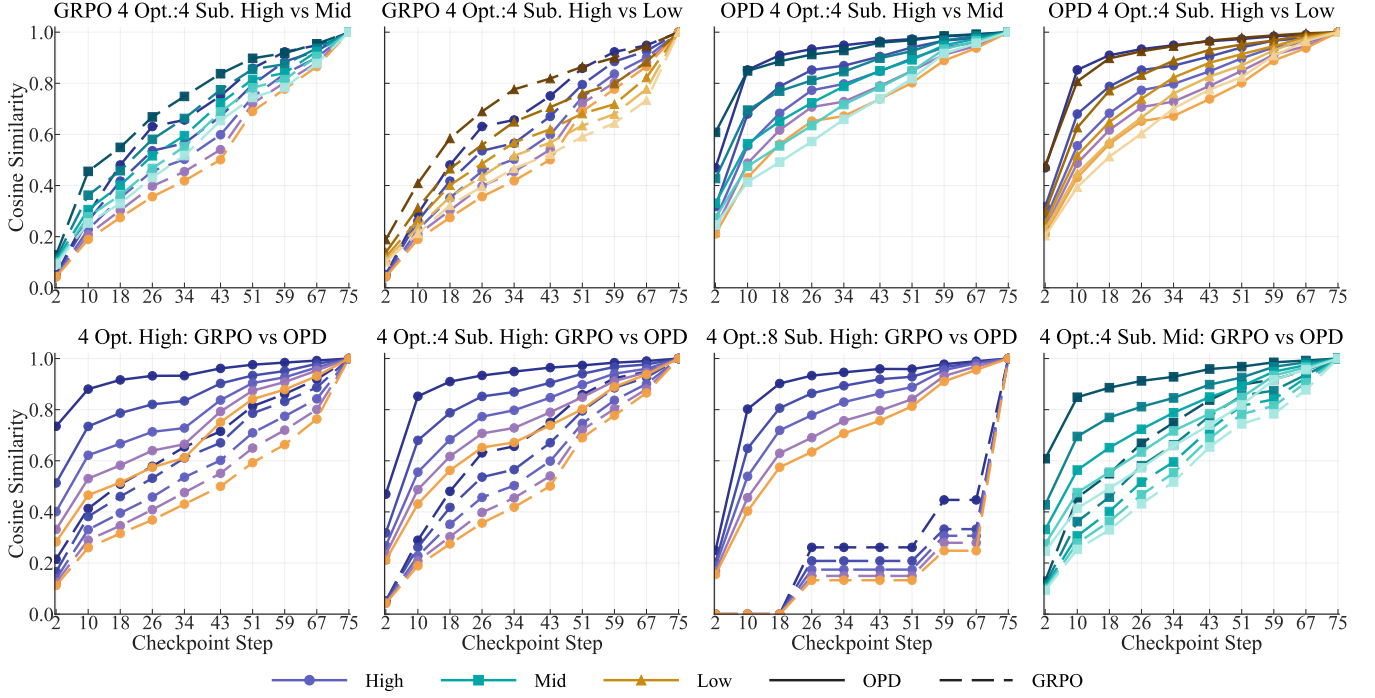


Figure 10: Evolution of parameter update directions. Top-5 SVD direction cosine similarity between intermediate and final checkpoints.

For each $\Delta W_\ell^{(t)}$, we compute SVD:

$$\Delta W_\ell^{(t)} = U_\ell^{(t)} \Sigma_\ell^{(t)} \left(V_\ell^{(t)} \right)^\top.$$

We keep the first k left singular vectors,

$$U_{\ell,k}^{(t)} = [u_{\ell,1}^{(t)}, u_{\ell,2}^{(t)}, \dots, u_{\ell,k}^{(t)}].$$

Given a target final checkpoint f , we compare checkpoint t with f by cosine similarity of matched singular vectors:

$$s_{\ell,i}(t, f) = \left| \frac{\left(u_{\ell,i}^{(t)} \right)^\top u_{\ell,i}^{(f)}}{\|u_{\ell,i}^{(t)}\|_2 \|u_{\ell,i}^{(f)}\|_2 + \varepsilon} \right|, \quad i = 1, \dots, k.$$

We use absolute value because SVD vectors can flip sign. The layer-level Top- k score is

$$s_\ell^{(k)}(t, f) = \frac{1}{k} \sum_{i=1}^k s_{\ell,i}(t, f).$$

Finally, we average over all valid layers $\mathcal{L}$:

$$S^{(k)}(t, f) = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} s_\ell^{(k)}(t, f).$$

$S^{(k)}(t, f) \in [0, 1]$; a larger value means the update direction at checkpoint t is closer to the final update direction. A larger value suggests that training has already entered a parameter-update direction similar to the final solution.

Experimental Setup. (1) **Horizon length.** Under the 4 Opt.:4 Sub setting, we evaluate both the GRPO and OPD algorithms. For each configuration, we conduct pairwise comparisons across the High, Mid, and Low difficulty levels. (2) **Algorithm comparison.** We fix the difficulty at the High level and utilize three settings: 4 Opt., 4 Opt.:4 Sub, and 4 Opt.:8 Sub. This part primarily compares the parameter update directions between the GRPO and OPD algorithms under high difficulty conditions. (3) **Top- k Subspace.** We select $k = 5$ for the top- k subspace analysis. For every layer, we compute the parameters of both the FFN and Attention modules. The maximum number of training steps is set to 75. We select step 1 as the initial checkpoint and uniformly sample 10 intermediate checkpoints for evaluation.

Results. As shown in Figure 9 and Figure 10. (1) **Coarse credit assignment in long horizons.** The first two plots in the first row display the performance of the GRPO algorithm across different difficulty levels. The plotted curves visually demonstrate that as the task difficulty increases, the numerical values remain notably lower. This directly indicates that as the horizon becomes longer, the coarse granularity of credit assignment causes the parameter update direction of GRPO to become inconsistent. (2) **Vanishing gradients with increasing non-optimal actions.** Observing the subfigures in the second row, specific GRPO curves begin exactly at zero and remain completely flat along the x-axis during the initial checkpoint steps. This distinct flat trajectory at zero provides evidence that continuously increasing the number of non-optimal actions triggers a vanishing gradient issue. Because of this issue, the GRPO gradients fail to undergo any updates. (3) **Superior directional consistency of OPD.** The second row of subfigures directly compares the GRPO and OPD algorithms across multiple configurations. The charts illustrate that the GRPO curves only reach higher values under restricted simple conditions. This proves that the update direction of GRPO is not universally consistent, relies heavily on specific applicability conditions, and is unsuitable for real long horizon scenarios. In contrast, the OPD curves maintain a relatively high level across all evaluated settings. This confirms that OPD maintains a much more consistent parameter update direction.

Takeaway

OPD exhibits stronger directional stability than GRPO under long-horizon and high-difficulty settings. While GRPO shows inconsistent or even vanishing update directions as task horizon and non-optimal actions increase, OPD maintains consistently high subspace alignment, suggesting more reliable and robust credit assignment in complex optimization scenarios.

Discussion

GRPO’s gradient consistency is conditional: unsuitable for multi-turn agents unlike single-step reasoning. The SVD analysis reveals that the directional consistency of GRPO is highly conditional. While coarse credit assignment may suffice for single-step mathematical reasoning tasks, it fundamentally breaks down in multi-turn, long-horizon agent scenarios. The flat zero-similarity curves indicate that accumulating non-optimal actions prevents the model from aligning with the correct low-rank update subspace. Consequently, in such complex multi-turn environments, optimization strategies that attempt to directly amplify parameter updates relying on early gradients will face severe limitations, as these early directions lack meaningful alignment.

Generalization demands cross-sample alignment: models effectively generalize to unseen data when principal gradient directions remain structurally aligned across samples. OPD successfully maintains high cosine similarity in its dominant left singular vectors ($U_{\ell,k}^{(t)}$) across all difficulty levels. This demonstrates that granular, step-level credit assignment is crucial for robustly stabilizing these principal update directions. This cross-sample consistency extends beyond mere optimization stability; it is the underlying mathematical mechanism for generalization. By ensuring that the principal gradient directions maintain structural alignment across various samples, OPD equips the model with the structural coherence required to effectively generalize to unseen data that shares similar gradient profiles.

Low mutual information patterns generalize better and OPD shows stronger capability in long tasks. Pretraining learns next word prediction on a mix of high and low quality data. This training objective differs from an agent predicting the next action, where success depends on the actions following the chain of thought. We view Reinforcement Learning as a process that stimulates the model based on environment rewards (GRPO) or teacher priors (OPD). We assume there is a low mutual information thinking pattern. When this low mutual information pattern correlates with the correct answers learned during the pretraining phase, optimizing this mutual information can lead to generalization. OPD has a much stronger capability than GRPO to stimulate these critical patterns in long horizon tasks. However, the performance of OPD is limited by the token overlap rate between the teacher and the student. Finally, the level of mutual information is relative. Compared to specific planning knowledge, planning patterns have lower mutual information. If a piece of planning knowledge applies to multiple samples, optimizing this process knowledge is also easier to generalize compared to other irregular tokens.

5.3. Impact of Planning Knowledge

5.3.1. Accuracy Analysis

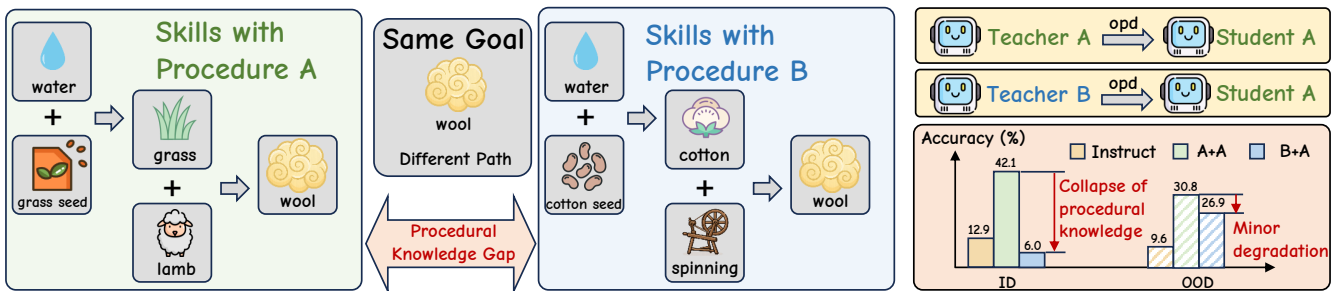


Figure 11: The procedural knowledge gap during teacher-student distillation. Distilling a different procedural path for the same goal via the OPD method (Teacher B to Student A) leads to a substantial collapse of procedural knowledge and decreased in-distribution accuracy.

Knowledge Gap in OPD. In real-world environments, there are multiple reasonable and effective paths to complete a task (e.g., Recipe A and Recipe B). This multi-path characteristic exposes a core issue in knowledge transfer: when the student and teacher models form divergent procedural knowledge based on different pre-training corpora, can this knowledge be effectively transferred? To this end, by comparing the mismatched setting (Student A + Teacher B) with the aligned setting (Student A + Teacher A), we investigate whether two completely different sets of procedural knowledge can successfully bridge this significant gap and be distilled. Specifically, we explore the scenario where the

student model conducts procedural knowledge world modeling based on Recipe A, while the teacher model provides procedural knowledge world modeling based on Recipe B. As shown in Figure 11, we illustrate this gap using a specific crafting example where both procedures share the same final goal of producing wool. Procedure A creates wool using water, grass seed, and a lamb. Procedure B achieves the identical goal using water, cotton seed, and spinning. When Student A, who is pre-trained on Procedure A, receives distillation from Teacher B via the OPD method, a severe collapse of procedural knowledge occurs. This structural mismatch leads to a significant drop in in-distribution accuracy compared to the aligned setting where Teacher A is used.

Task Setup. (1) **Models.** For the base model, we use 4 Opt.:4 Sub. For the teacher models, we train them using Recipe A and Recipe B. Although these two recipes use different raw materials and input materials for the same target, both can successfully make the target. (2) **Training.** We train the models on the Mid reinforcement learning training set. The subset uses FA.

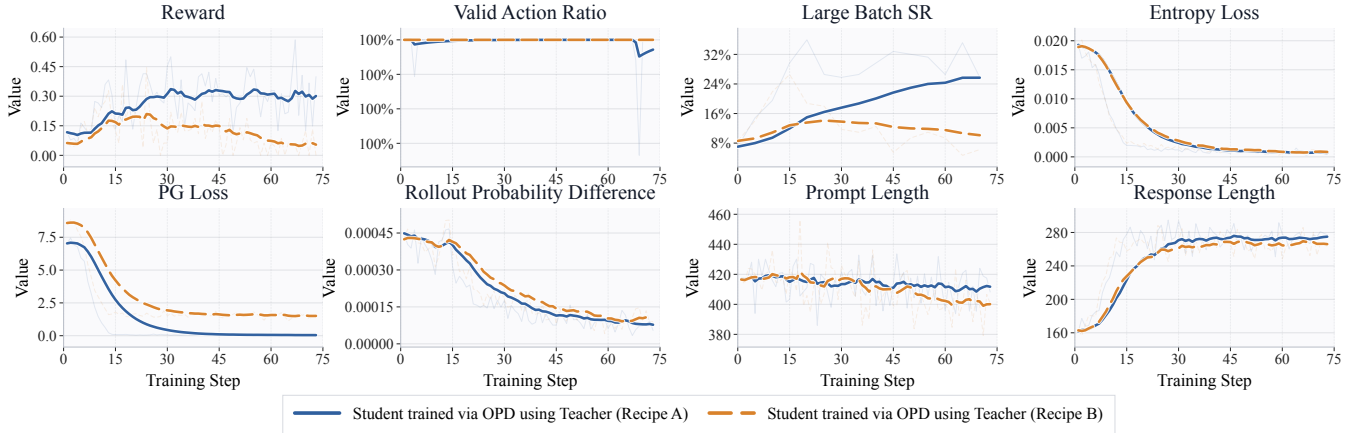


Figure 12: Comparison of training dynamics under two OPD teacher recipes.

Table 4: Pretrain gap comparison for the 4 Opt.:4 Sub. pattern mixture; the final Δ row reports Student + Teacher B minus Student + Teacher A.

Pattern Mix	Model	Low				Middle				High				Avg	
		FA	LF	EA	Mean	FA	LF	EA	Mean	FA	LF	EA	Mean	L/M/H Mean (Δ)	
AVG@8															
4 Opt.:4 Sub.	Instruct Model	75.00	61.41	67.97	68.12	12.97	9.69	5.31	9.32	1.56	0.39	0.86	0.94	26.13	$\Delta +0.00$
	Teacher A	100.00	–	–	100.00	100.00	–	–	100.00	100.00	–	–	100.00	100.00	$\Delta +73.87$
	Teacher B	100.00	–	–	100.00	100.00	–	–	100.00	100.00	–	–	100.00	100.00	$\Delta +73.87$
	Student + Teacher A	91.25	71.56	79.77	80.86	42.19	30.86	26.48	33.18	12.89	6.41	10.94	10.08	41.37	$\Delta +15.24$
	Student + Teacher B	72.19	63.91	78.75	71.61	6.09	26.95	20.78	17.94	2.27	5.31	7.27	4.95	31.50	$\Delta +5.37$
	Δ (B - A)	-19.06	-7.66	-1.02	-9.24	-36.09	-3.91	-5.70	-15.23	-10.62	-1.09	-3.67	-5.13		-9.87
PASS@8															
4 Opt.:4 Sub.	Instruct Model	95.62	83.75	88.75	89.38	45.62	38.75	25.62	36.67	9.38	2.50	6.25	6.04	44.03	$\Delta +0.00$
	Teacher A	100.00	–	–	100.00	100.00	–	–	100.00	100.00	–	–	100.00	100.00	$\Delta +55.97$
	Teacher B	100.00	–	–	100.00	100.00	–	–	100.00	100.00	–	–	100.00	100.00	$\Delta +55.97$
	Student + Teacher A	92.50	77.50	85.00	85.00	51.25	36.88	34.38	40.83	17.50	10.00	15.62	14.38	46.74	$\Delta +2.71$
	Student + Teacher B	75.62	73.75	85.00	78.12	7.50	34.38	29.38	23.75	3.75	8.75	11.25	7.92	36.60	$\Delta -7.43$
	Δ (B - A)	-16.88	-3.75	+0.00	-6.88	-43.75	-2.50	-5.00	-17.08	-13.75	-1.25	-4.38	-6.46		-10.14

Results. As shown in Figure 12 and Table 4. (1) **Knowledge mismatch blocks transfer even with perfect teachers.** Teacher B achieves a perfect score of 100.00 in all tasks. However, training a student with a different recipe fails completely. The Student + Teacher B model gets an overall PASS@8 score of only 36.60. This is worse than the base Instruct Model score of 44.03. This shows that a highly capable teacher cannot improve the student if their training recipes do not match. (2) **Unmatched procedural knowledge fails to transfer and destroys original modeling.** The

performance drop from using different recipes is very large on certain subsets. In the Middle difficulty FA task, the Student + Teacher B model performs much worse than the Student + Teacher A model. The AVG@8 score drops by 36.09 and the PASS@8 score drops by 43.75. This shows that the different procedural knowledge from Teacher B cannot be transferred to the student. Instead, it actively destroys the original world modeling of the student. (3) **Large knowledge gap causes training collapse in later stages.** Figure 12 shows that the performance goes up in the early stage because the student aligns the planning pattern. In the later stage, the main optimization direction is to align the planning knowledge. However, the knowledge gap between the two recipes is too big, which finally leads to a complete training collapse.

Takeaway

OPD requires aligned procedural knowledge between student and teacher. OPD cannot transfer knowledge from a perfect teacher if there is a gap in internalized procedural knowledge between the student and the teacher. Both models must share similar internal knowledge for successful distillation.

Forced OPD severely damages specific planning abilities. Applying OPD across unmatched knowledge actively destroys the original world modeling required for complex planning tasks. However, it hardly affects other basic tasks that do not rely on this specific procedural knowledge.

OPD distills patterns early but fails on mismatched knowledge late. In the early stage, OPD successfully distills better planning patterns to improve performance. In the later stage, forcing OPD to distill completely different procedural knowledge leads to a training collapse.

5.3.2. KL Dynamic Analysis

KL Dynamic and Overlap. We use KL divergence to measure how close a trained student is to one of the two teacher models at the token distribution level. Let T_A and T_B denote the two teacher models. All student checkpoints start from the same T_A initialization. We use $\sigma \in \{A, B\}$ to denote which teacher is used to train the student, and $\gamma \in \{A, B\}$ to denote which teacher is used for KL comparison. The student trained with teacher T_σ is denoted as π_{θ_σ} . A trajectory $\tau \sim \pi_{\theta_\sigma}$ is rolled out by this student. On this fixed trajectory, the student policy $p_{k,t}^\sigma$ and the comparison teacher policy $q_{k,t}^\gamma$ evaluate next-token probabilities at decoding step t within turn k , conditioned on the same prefix. Let $\mathcal{V}_{k,t}$ be the top- k token subset, and let $\ell_{k,t}^{p,\sigma}(v)$ and $\ell_{k,t}^{q,\gamma}(v)$ be the stored log probabilities for token v .

$$\begin{aligned} p_{k,t}^\sigma(v) &= \exp\left(\ell_{k,t}^{p,\sigma}(v)\right), \quad q_{k,t}^\gamma(v) = \exp\left(\ell_{k,t}^{q,\gamma}(v)\right), \\ p_{k,t,\text{tail}}^\sigma &= 1 - \sum_{v \in \mathcal{V}_{k,t}} p_{k,t}^\sigma(v), \quad q_{k,t,\text{tail}}^\gamma = 1 - \sum_{v \in \mathcal{V}_{k,t}} q_{k,t}^\gamma(v). \end{aligned} \quad (8)$$

For each token position, we compute the KL from the student distribution to the selected teacher distribution, and average it over the whole trajectory. We use four comparison settings: setting 1 uses the student trained with T_A and compares it with T_A ; setting 2 uses the student trained with T_B and compares it with T_B ; setting 3 uses the student trained with T_A and compares it with T_B ; setting 4 uses the student trained with T_B and compares it with T_A .

$$\text{KL}^{\sigma,\gamma}(\tau) = \frac{1}{\sum_{k=1}^K T_k} \sum_{k=1}^K \sum_{t=1}^{T_k} \sum_{v \in \mathcal{V}_{k,t} \cup \{\text{tail}\}} P_{k,t}^\sigma(v) \log \frac{P_{k,t}^\sigma(v)}{Q_{k,t}^\gamma(v)}. \quad (9)$$

We further split trajectories into four data types according to trajectory correctness and step correctness: correct trajectory with correct step, correct trajectory with incorrect step, incorrect trajectory with correct step, and incorrect trajectory with incorrect step. Let $\mathcal{R}_c$ be the set of trajectories in data type c . For each training step, comparison setting (σ, γ) , and data type c , we report the mean KL. To compare two data types c and c' , we estimate their KL density functions $f_c(x)$ and $f_{c'}(x)$ with Gaussian kernel density estimation, where x is a trajectory-level KL value, and compute their overlap.

$$\overline{\text{KL}}_c^{\sigma,\gamma} = \frac{1}{|\mathcal{R}_c|} \sum_{\tau \in \mathcal{R}_c} \text{KL}^{\sigma,\gamma}(\tau), \quad \text{Overlap}(c, c') = \int \min(f_c(x), f_{c'}(x)) dx. \quad (10)$$

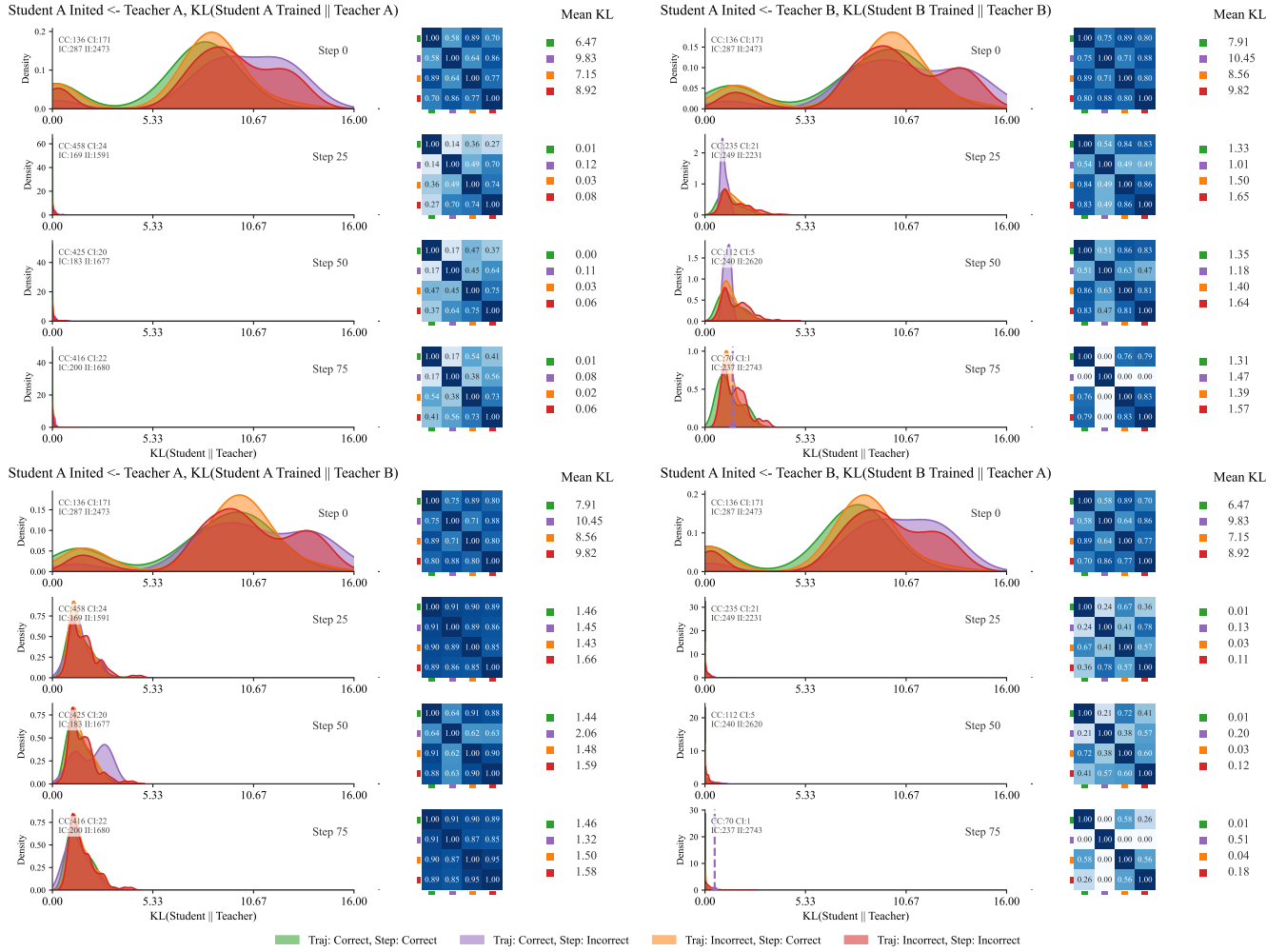


Figure 13: Token-level KL divergence distributions between students and matched or crossed teachers across training steps. Samples are grouped by trajectory-level and step-level correctness.

Task Setup. (1) **Data and model inference.** We use the two models trained in the previous section to perform inference on the FA test set with Mid difficulty. The rollout process uses two models pretrained from recipe A and trained later by teacher A and teacher B respectively. Then we calculate the reverse KL divergence. We also divide the rollout data into four types based on trajectory correctness and single step correctness to obtain the distribution, overlap rate, and mean of the KL divergence. (2) **Distribution recording and divergence calculation.** During the KL divergence calculation stage, the models perform a rollout independently and record the probability distribution. Subsequently, teacher A and teacher B record the output probability distributions on these rollout trajectories and calculate the reverse KL divergence based on this. (3) **Data classification and statistical settings.** We divide the rollout data into four types based on trajectory correctness and single step correctness to obtain the distribution, overlap rate, and mean of the KL divergence. At the same time, we set the K value of top K to 100 and calculate the RL training dynamics at four points, which are 0, 25, 50, and 75.

Results. As shown in Figure 13 and Figure 14 and Figure 15. (1) **Need for error correction data.** The proportion of the four data types shows that the number of correct trajectories with incorrect steps is extremely small. This indicates a need to add more error correction data. Gold standard data alone fails to stimulate the recovery ability from error state prefixes. (2) **Optimization of planning knowledge.** As shown in the first group of figures, the KL divergence on the left side is successfully optimized down. In contrast, after optimizing the planning patterns, the KL divergence on the right side becomes difficult to decrease further. Planning knowledge has large token mutual

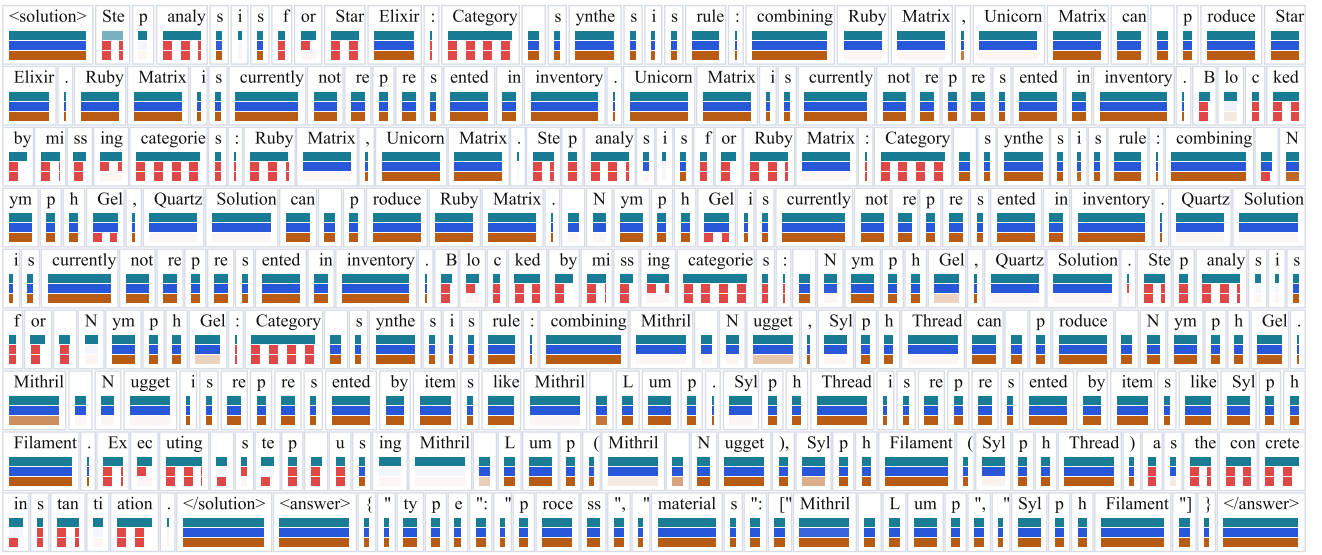


Figure 14: Token-level probability heatmap on a Student A rollout. For each token generated by Student A, the top cyan, middle blue, and bottom orange bars show the probabilities assigned by Student A, Teacher A, and Teacher B, respectively; darker colors indicate higher probabilities. Red striped bars indicate that the generated token is absent from the corresponding teacher’s top-k predictions.

information and requires point by point optimization. The reinforcement learning update magnitude is too small and is insufficient to transfer massive amounts of knowledge to the domain of teacher B. (3)**Distribution discrimination.** The KL divergence distribution shows high discrimination when the planning knowledge of the teacher and the student matches. Conversely, the distribution discrimination is low when their planning knowledge mismatches. (4)**Reward correlation and knowledge disruption.** Based on the average KL values, the mean KL divergence for many correct steps is larger than that for incorrect steps. The correlation between the mean KL divergence and the environment reward is smaller for mismatched planning knowledge compared to matched planning knowledge. Completing a single sample involves multiple reasoning paths. If the teacher fails to cover these paths, the model treats other correct paths as incorrect ones. This process destroys the original knowledge of the model. (5)**Transfer failure in later stages.** As shown in Figure 14 and Figure 15, both models optimize the planning pattern in the early stages. In the later stages, teacher B primarily optimizes planning knowledge. However, planning knowledge varies greatly across different samples. The reinforcement learning update fails to transfer from knowledge A to knowledge B.

Takeaway

Incorporate error correction data. Relying solely on gold standard data is insufficient. Training pipelines need error state prefixes to teach models how to recover from mistakes.

Acknowledge RL limits in knowledge transfer. Standard reinforcement learning updates are too small for the point by point optimization required to transfer massive planning knowledge between domains.

Match teacher and student knowledge. KL divergence effectively discriminates distributions only when the teacher and student share underlying planning knowledge.

Cover multiple reasoning paths. If the teacher model fails to cover alternative correct paths, it penalizes valid reasoning and destroys the pre existing knowledge of the student.

Separate pattern learning from knowledge learning. Models easily optimize general planning patterns early in training, but they struggle to transfer high variance planning knowledge in later stages.

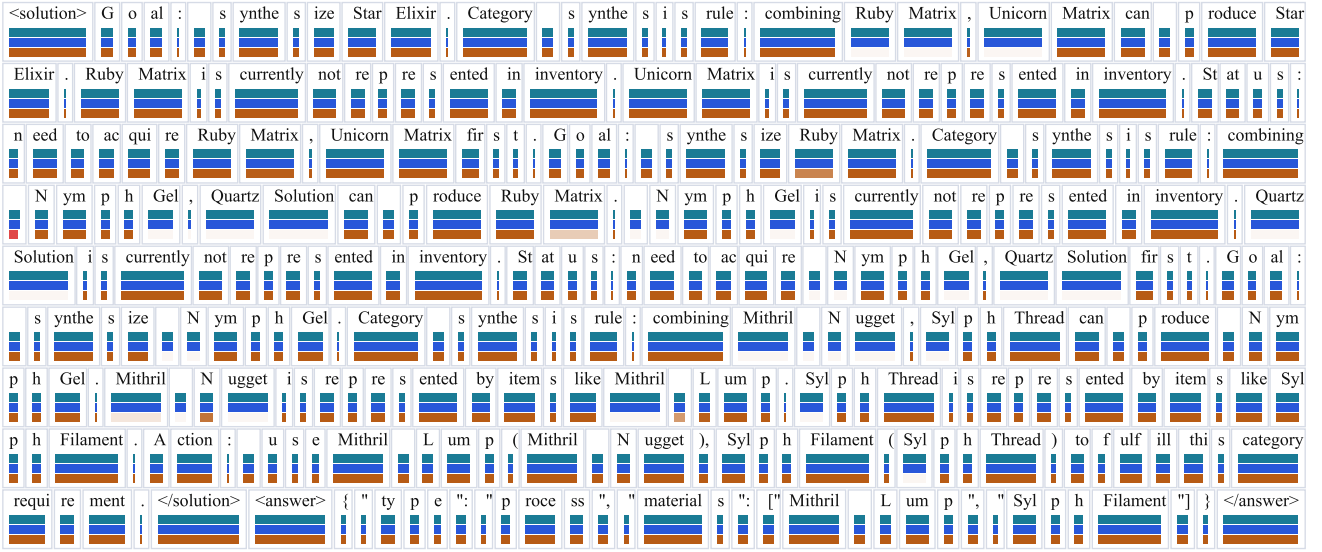


Figure 15: Token-level probability heatmap on a Student A rollout for step 75.

6. Multi Teachers On-Policy Agentic Distillation

6.1. Key Concepts and Research Problems

Multi-Teacher On-Policy Agentic Distillation (MOPD) is a method that distills and combines the abilities of multiple expert models into one student model. Existing technical route mainly follow two paradigms, as shown in Table 5. (1) **Cross domain capability integration.** This paradigm trains the student model in different environments based on the student model. After training, teacher models from different environments are merged to integrate the abilities of multiple experts (Xiao et al., 2026, Xu et al., 2026). (2) **Cross stage capability retention.** This paradigm uses curriculum learning to train the student model. The model learns from reasoning ability to agent ability step by step. In the final stage, the final model from each stage is used as a teacher model and then merged to recover abilities that are forgotten in earlier stages (Zeng et al., 2026, Yang et al., 2026c).

Table 5: Comparison of MOPD paradigms across different foundation models.

Foundation Models	Training Paradigm	Student	Teacher	Primary Motivation
MiMo-V2-Flash (Xiao et al., 2026)	Pre-training → Domain training → MOPD	SFT Model	Domain-specialized Model	Cross domain capability integration
GLM-5 (Zeng et al., 2026)	Pre-training → Mid-training → Cascading RL → MOPD	Last RL checkpoint	Previous RL checkpoint	Cross stage capability retention
Nemotron-Cascade 2 (Yang et al., 2026c)	Pre-training → Multi-domain RL → MOPD → Cascading RL	Middle RL checkpoint	Previous RL checkpoint	Cross stage capability retention
DeepSeek-V4 (Xu et al., 2026)	Pre-training → Domain training → MOPD	SFT Model	Domain-specialized Model	Cross domain capability integration

Multi-teacher agent-OPD can generally be categorized into two paradigms: mixture and cascaded. Given that the mixture approach incurs infrastructure overhead (e.g., simultaneous deployment and inference of multiple teacher

models), we focus on the cascaded form. Under this sequential training paradigm, the student policy learns from one teacher at a time. For the m -th teacher ($m \in \{1, \dots, M\}$), the objective provides dense, step-by-step supervision by aligning the student’s predictions with those of the specific teacher:

$$\mathcal{L}_{\text{Agent-OPD}}^{(m)}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{k=1}^K \sum_{t=1}^{T_k} D_{\text{KL}}(p_{k,t} \parallel q_{k,t}^{(m)}) \right], \quad (11)$$

where $q_{k,t}^{(m)} \triangleq \pi_{\text{teacher}}^{(m)}(\cdot \mid H_k, \hat{Y}_{k,<t})$ denotes the next-token distribution provided by the m -th teacher. The overall training proceeds by iteratively minimizing $\mathcal{L}_{\text{Agent-OPD}}^{(m)}(\theta)$ in sequence from $m = 1$ to M .

Although MOPD is widely used in existing foundation models, the training mechanisms and conditions for MOPD remain unclear. Therefore, we urgently need to analyze the MOPD algorithm deeply to explore its working conditions and boundaries. Thus, we propose three progressive research questions:

- **Q1:** Under what conditions does MOPD effectively achieve generalization across environments?
- **Q2:** Under what conditions does MOPD lose generalization across environments but still maintain a state without conflicts among environments?
- **Q3:** Under what conditions does MOPD lose generalization across environments and cause severe conflicts among environments?

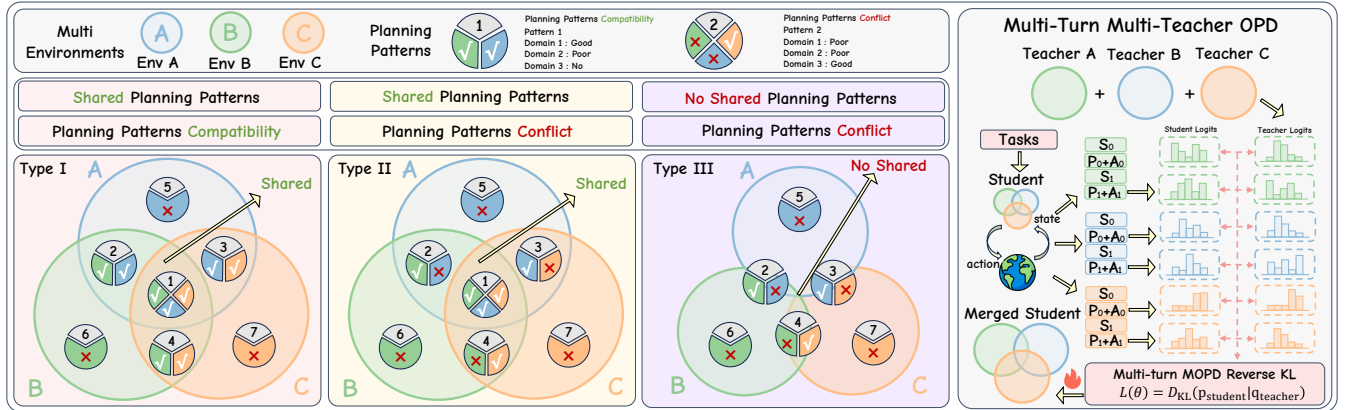


Figure 16: Multi-Turn MOPD. Analysis framework of non-shared and conflicting planning patterns in different environments for student policy optimization.

To analyze the three problems above, we propose a unified framework. As shown in Figure 16, we use two dimensions for our analysis: (1) whether there is a shared area of planning patterns, and (2) whether the planning patterns are compatible with different environments. Based on these two dimensions, we classify the relationship between planning patterns and environments into three typical coverage patterns: Type I (Shared and Compatibility Planning Patterns), Type II (Shared and Conflict Planning Patterns), and Type III (No Shared and Conflict Planning Patterns). These structural relationships determine how the planning patterns distribute between the student and multiple expert teachers across different environments.

6.2. Shared and Compatibility Multi Teachers On-Policy Agent Distillation

To address the first research question (Q1), we focus on the **Type I** category: **Shared and Compatibility Planning Patterns**. In this scenario, different environments share common planning patterns that are entirely compatible and free of cross-domain conflicts. This implies that the student and the multiple expert teachers possess an intersecting and consistent understanding of the tasks across different domains. To simulate this structural relationship and empirically evaluate how MOPD achieves effective cross-environment generalization under these conditions, we design the following experimental setup.

Table 6: Ensemble main table. Rows are grouped by pattern mixture. The teacher group reports Teacher FA, Teacher LF, and Teacher EA. The other groups report Instruct, After FA, After LF, and After EA, and are filled with 0.00 when no result is available. Two global blocks are reported: Avg@8 and Pass@8. Domain abbreviations: FA = Fantasy Alchemy; LF = Livestock Farming; EA = Electronic Assembly.

Pattern Mix	Model	Low				Middle				High				Avg
		FA	LF	EA	Mean	FA	LF	EA	Mean	FA	LF	EA	Mean	L/M/H Mean
AVG@8														
Teacher	Teacher FA	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	33.33
	Teacher LF	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	33.33
	Teacher EA	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	33.33
4 Opt.:4 Sub.(a)	Instruct	86.25	80.70	79.45	82.14	41.95	34.92	30.08	35.65	13.83	8.83	8.91	10.52	42.77
	After FA	92.50	88.59	86.41	89.17	61.64	52.58	48.59	54.27	31.64	24.38	22.19	26.07	56.50
	After LF	90.55	92.81	85.55	89.64	60.55	58.36	48.75	55.89	28.75	27.66	28.44	28.28	57.93
	After EA	88.59	92.73	85.08	88.80	51.95	47.42	47.81	49.06	22.50	18.83	27.81	23.05	53.64
4 Opt.:4 Sub.(b)	Instruct	77.19	62.66	68.44	69.43	14.38	9.53	5.23	9.71	1.17	0.23	0.70	0.70	26.61
	After FA	91.02	73.59	81.25	81.95	45.08	36.95	25.16	35.73	15.47	5.39	12.27	11.04	42.91
	After LF	90.00	73.05	80.62	81.22	45.16	33.36	26.02	34.84	15.23	6.88	13.44	11.85	42.64
	After EA	85.00	69.69	79.53	78.07	37.42	30.62	21.56	29.87	10.00	6.33	13.05	9.79	39.24
4 Opt.:8 Sub.	Instruct	51.64	40.55	42.27	44.82	0.00	0.08	0.00	0.03	0.00	0.00	0.00	0.00	14.95
	After FA	84.06	62.58	72.58	73.07	30.23	21.72	17.03	22.99	7.66	3.67	5.16	5.49	33.85
	After LF	83.83	61.09	71.48	72.14	28.83	21.17	18.12	22.71	7.81	3.36	5.00	5.39	33.41
	After EA	81.80	60.55	66.56	69.64	26.33	16.88	15.55	19.58	6.41	1.88	3.05	3.78	31.00
PASS@8														
Teacher	Teacher FA	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	33.33
	Teacher LF	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	33.33
	Teacher EA	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	33.33
4 Opt.:4 Sub.(a)	Instruct	98.12	96.25	92.50	95.62	75.62	66.88	63.12	68.54	40.62	32.50	30.62	34.58	66.25
	After FA	95.00	95.62	93.12	94.58	68.12	66.88	65.62	66.88	41.25	40.62	40.62	40.83	67.43
	After LF	95.62	95.00	90.00	93.54	68.75	65.62	56.88	63.75	39.38	38.12	36.25	37.92	65.07
	After EA	91.88	95.62	90.62	92.71	60.62	58.75	58.12	59.17	32.50	28.75	41.88	34.38	62.08
4 Opt.:4 Sub.(b)	Instruct	95.62	86.88	89.38	90.62	51.25	38.12	28.12	39.17	8.12	1.88	5.00	5.00	44.93
	After FA	91.88	80.00	85.62	85.83	50.62	44.38	33.75	42.92	21.25	10.00	18.75	16.67	48.47
	After LF	91.88	78.75	88.12	86.25	51.88	41.88	36.25	43.33	21.88	10.00	21.25	17.71	49.10
	After EA	88.12	76.25	85.00	83.12	45.00	40.00	32.50	39.17	16.25	8.75	20.62	15.21	45.83
4 Opt.:8 Sub.	Instruct	70.62	61.25	67.50	66.46	0.00	0.62	0.00	0.21	0.00	0.00	0.00	0.00	22.22
	After FA	89.38	68.75	78.75	78.96	37.50	29.38	26.88	31.25	12.50	5.00	7.50	8.33	39.51
	After LF	88.12	68.75	78.75	78.54	34.38	28.75	26.25	29.79	10.62	6.88	8.12	8.54	38.96
	After EA	87.50	66.88	73.12	75.83	35.00	25.62	21.88	27.50	10.00	4.38	6.25	6.88	36.74

Task Setup. (1) **Teacher models.** Each teacher model focuses on one environment. The experiment includes three domains: fantasy alchemy, livestock farming, and electronic assembly. We train an expert teacher model for each domain, so we get three domain expert teachers in total. Each teacher model learns from the gold trajectories in the pretraining and posttraining datasets of its domain. To bring in different planning supervision signals, we design four planning patterns for internal world models. Three expert teachers share one planning template. We assign the other templates to only some teachers. We distribute three planning patterns equally for each teacher. Therefore, all three teachers learn the correct trajectory supervision, but they see different planning template distributions and have varied planning patterns. (2) **Student models.** The student models use 4 Opt.:4 Sub. and 4 Opt.:8 Sub. as base models. The 4 Opt.:4 Sub. setting has two versions: 4 Opt.:4 Sub.(a) and 4 Opt.:4 Sub.(b). For the student SFT data, we build two group settings on the same three domains: electronic assembly, fantasy alchemy, and livestock farming. Both settings build on pretraining trajectories and include two kinds of supervision signals: normal planning samples from gold actions, and shortcut style distractor samples with wrong actions. Both settings use four kinds of normal planning templates and four kinds of distractor planning templates. We make the sample size of each template type equal to keep an overall balance between normal samples and distractor samples. The main difference between the two settings is the domain and template distribution of normal planning samples. Form a uses a mixed distribution: the three

domains share one normal planning template, and the other normal planning templates only appear in two domains. Therefore, the subsets of planning patterns seen by different domains are not exactly the same. Form b removes this difference: every normal planning template appears in all three domains with equal sampling. Thus, Form a focuses on the unequal planning pattern exposure among domain experts. Form b makes sure each domain covers all planning templates, while keeping the same domain scope and equal distractor construction. (3) **Training configurations.** To ensure a strict comparison, the three student models use the exact same amount of pretraining data and the same training parameters. The student models go through OPD one by one in the order of FA, LF, and EA. We train the OPD on the posttraining dataset with a middle difficulty level.

Results. As shown in Table 6. (1) **Shared and compatible planning patterns are a basic condition for cross environment generalization.** Table 6 shows that although teacher models only work in their own domains (for example, Teacher FA has an Avg@8 of 100.00 in the FA domain but 0.00 in LF and EA), student models achieve significant performance improvements in unseen domains after distillation from a single domain teacher. Take 4 Opt.:4 Sub.(a) as an example. In Middle difficulty, for in-domain environments, the Avg@8 of the base Instruct model in the FA domain increases from 41.95 to 61.64. For out of domain environments, the Avg@8 of the base Instruct model in the LF domain is 34.92. However, after it receives distillation only from the FA domain teacher (the After FA setting), its performance in the LF domain jumps directly to 52.58. This indicates that when different environments share compatible planning patterns, MOPD breaks domain barriers and allows the model to transfer abilities learned in one environment to other environments. (2) **Cross environment generalization does not directly depend on the percentage of optimal actions in the data.** We compare the 4 Opt.:4 Sub. model with the 4 Opt.:8 Sub. model that introduces more interfering options. The Instruct model of the latter has very low base performance because a large number of low quality trajectories affect it. Its average Avg@8 in Middle difficulty is only 0.03. However, after MOPD training, such as in the After EA setting, the performance of the 4 Opt.:8 Sub. model recovers greatly in all domains. Specifically, its average Avg@8 in Middle difficulty successfully increases to 19.58. This shows that as long as shared planning patterns exist at the bottom level of the model, MOPD effectively extracts them and achieves cross environment generalization. This is true even if a large amount of low quality data covers these high quality patterns during the SFT stage. The generalization effect does not strictly depend on the initial percentage of high quality data.

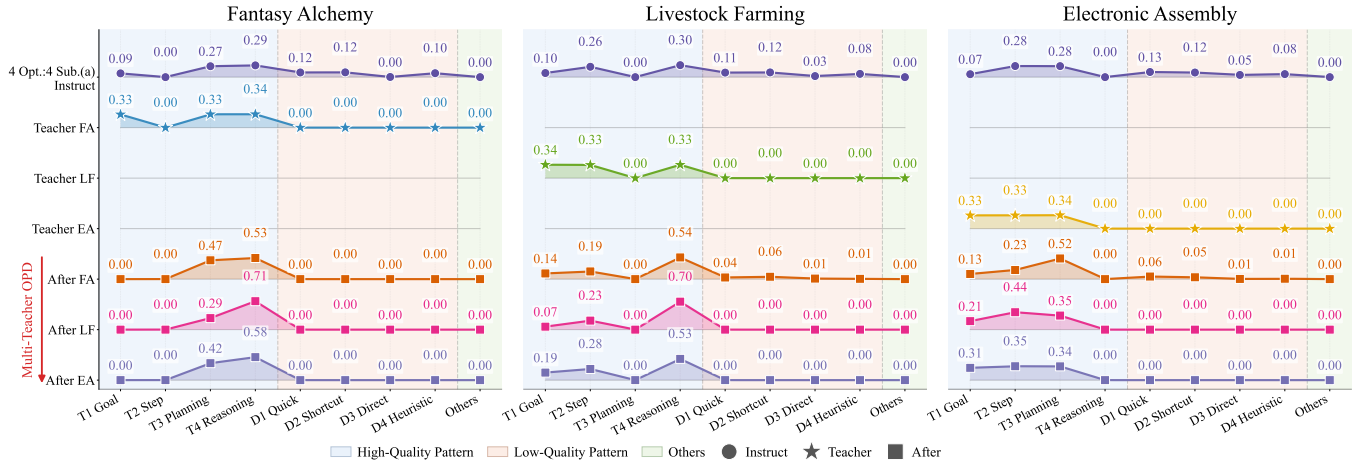


Figure 17: Planning pattern distributions for 4 Opt.:4 Sub. (a) before and after multi-teacher OPD.

Results. As shown in Figure 17, Figure 18, and Figure 19. (1) **Consistent with previous studies, the process is mode seeking rather than mode covering.** As the distribution change graph in Figure 18 shows, the student model in the Instruct stage presents a relatively uniform distribution across various high quality planning patterns. However, after MOPD training, the probability distribution of the model does not cover all teacher preferences. Instead, it quickly shrinks to a portion of the shared patterns. For example, in Figure 18, the trained model strongly focuses on the "T1 Goal" and "T4 Reasoning" patterns in all three environments, and it does not cover the T3 pattern of the teacher. This perfectly matches the mode seeking phenomenon of the reverse KL divergence in existing OPD. (2) **The core of MOPD is to find the common distribution among multiple teachers.** As shown in Figure 18 and Figure 19. Sequential distillation drives a consistent shift in the planning patterns of the student model across all domains. For

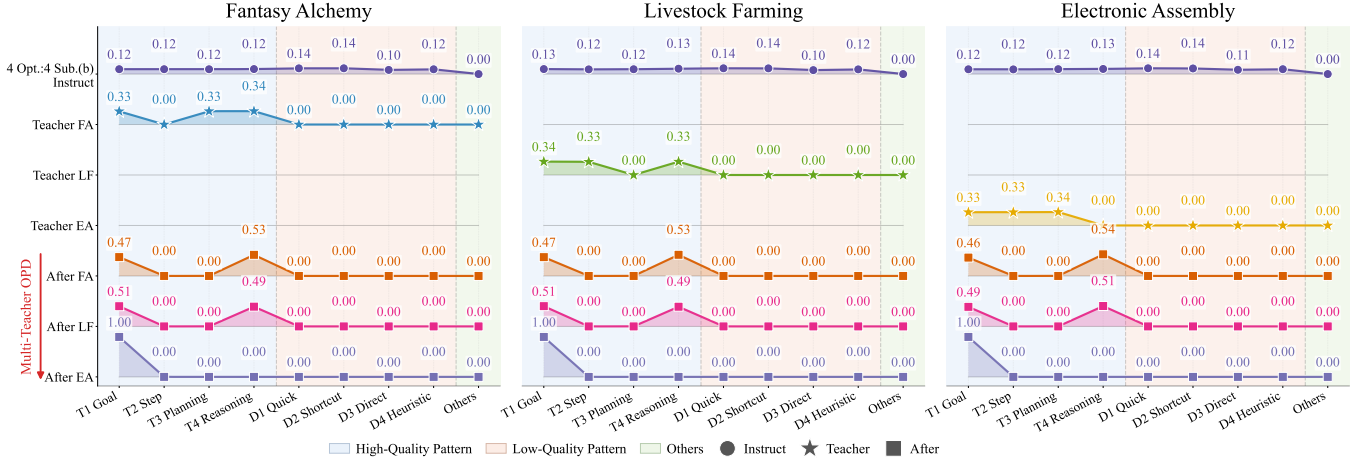


Figure 18: Planning pattern distributions for 4 Opt.:4 Sub. (b) before and after multi-teacher OPD.

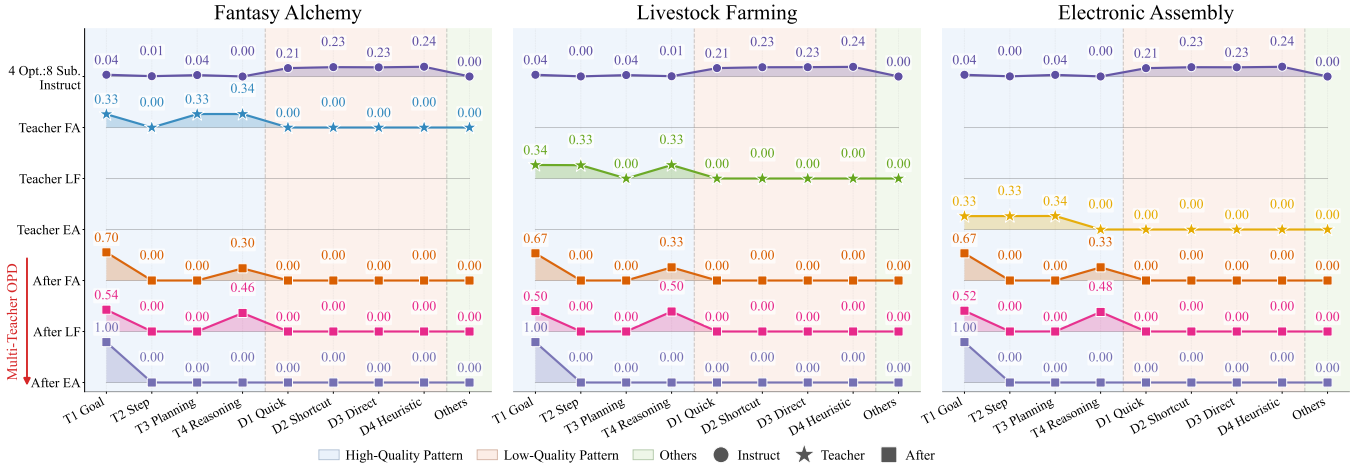


Figure 19: Planning pattern distributions for 4 Opt.:8 Sub. before and after multi-teacher OPD.

example, in Figure 18, when the student model completes the distillation from the last teacher (EA), its planning pattern distributions in Fantasy Alchemy, Livestock Farming, and Electronic Assembly all converge to the single "T1 Goal" template with a ratio of 1.00 at the same time. This shows that sequential MOPD does not cause separate local adaptations in different domains. Instead, it drives the model to find and lock onto a globally unified common distribution among the teacher strategies of each domain. (3)**Training on one environment affects the planning patterns of other environments that share the same distribution.** As shown in Figure 19, the experimental results in the first two columns reveal that the planning pattern trained in a single environment shows the same changes in other environments. For example, combining the data in the After FA row, when the model finishes training in the Fantasy Alchemy environment, the percentage of the T2 Step pattern in both the local FA environment and the unseen Livestock Farming (LF) environment shows the exact same decline. Specifically, both values drop from the baseline of 0.12 to 0.00. At the same time, the T4 Reasoning pattern in these two environments activates at the same time across domains, and both accurately reach 0.53. This indicates that as long as different environments share a consistent distribution basis in their underlying task structures, optimizing for a single environment directly causes cross domain generalization for the planning patterns in other environments. (4)**However, this method fails to trigger planning patterns that do not exist in the model's pretraining data for other environments.** This limitation is very clear in the T3 Planning changes in Figure 17. As the results in the After FA row show, when we introduce specific optimization in the Fantasy Alchemy environment, the model successfully increases the percentage of the T3 Planning pattern in the local FA environment from 0.27 in the Instruct stage to 0.47. However, when we apply this successful training signal

from the FA environment to the Livestock Farming (LF) environment, it fails to trigger cross environment generalization for this planning pattern. This is because this planning pattern does not exist in the pretraining data of Livestock Farming. This proves that OPD cross environment generalization cannot artificially create or trigger specific planning patterns that are missing in the pretraining data of the target environment.

Takeaway

Shared and compatible planning patterns enable cross-environment generalization. When multiple environments share compatible planning patterns, MOPD can transfer the capability learned from one domain teacher to other domains, even if each teacher itself is only effective in its own environment.

MOPD performs mode seeking over the shared teacher distribution. Sequential multi-teacher OPD does not simply cover all planning patterns from all teachers. Instead, it tends to collapse toward a subset of shared high-quality planning modes, revealing that the core mechanism of MOPD is to identify and amplify the shared distribution among teachers.

Cross-environment generalization is bounded by the student’s existing pattern support. MOPD can activate and strengthen planning patterns that already exist in the student’s underlying distribution, even when they are diluted by low-quality SFT data. However, it cannot reliably create planning patterns that are absent from the target environment’s pretraining or SFT support.

Discussion

Potential paths for sim-to-real generalization. Our results suggest that simulation training can generalize to the real world when the simulated and real environments share the same planning patterns. However, this transfer can still be limited by differences in observations between simulation and reality. This also implies that some knowledge used in simulation may already exist in the pretrained model, so simulation can activate such knowledge rather than create it from scratch. However, if simulation data is used for SFT, we may need to consider the differences in environment knowledge and process knowledge between simulation and reality. We leave a deeper study of these issues to future work.

6.3. Non-Shared Conflicting Multi Teachers On-Policy Agent Distillation

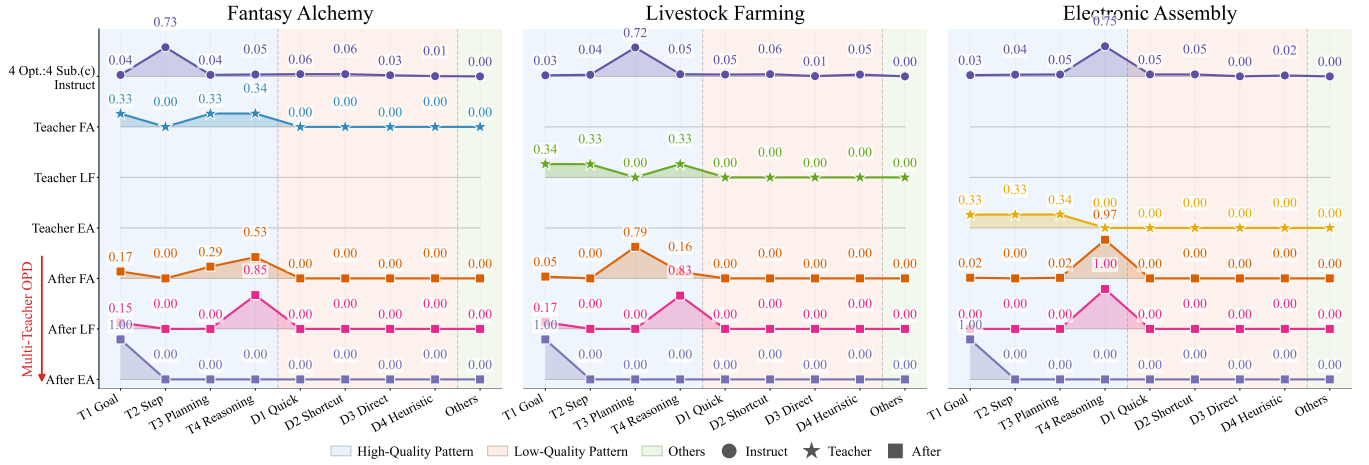
The previous section shows that MOPD generalizes across environments when teachers share compatible planning patterns. We now study the harder non-shared setting, where cross-environment generalization is no longer guaranteed. This section addresses both **Q2** and **Q3**: when non-shared teacher patterns remain compatible, MOPD may preserve domain-specific abilities without causing conflicts. When these patterns become incompatible, sequential distillation can overwrite earlier behaviors and induce severe cross-environment interference.

Task Setup. (1) **Student models.** The student models use Form c and Form d as two SFT settings on the same three domains: electronic assembly, fantasy alchemy, and livestock farming. Both settings build on pretraining trajectories and contain three supervision signals: normal planning samples with gold actions, shortcut style distractor samples with wrong actions, and targeted wrong answer samples whose reasoning trace is correct but final action is wrong. (2) **Form c.** It keeps the Form b style coverage: all four normal planning templates appear in all three domains with balanced domain sampling, and all four distractor templates are also used. It chooses three domain and template groups, template 2 in fantasy alchemy, template 3 in livestock farming, and template 4 in electronic assembly. For these groups, the answer is replaced by a wrong shortcut action while the reasoning trace remains correct. Each selected group uses three times the ordinary per-domain sample size. (3) **Form d.** It keeps the same three targeted wrong answer groups as Form c, with the same three times sampling. The main difference is that normal planning template 1 is removed. Only normal templates 2, 3, and 4 appear in all three domains. However, all four distractor templates are still kept. Thus, Form d tests missing normal template exposure while keeping full shortcut distractor coverage and the same targeted correct reasoning wrong answer signal.

Results. As shown in Table 7, Figure 20, and Figure 21. (1) **Compatible patterns allow sequential learning.** Under the Form c setting, the model successfully learns from different teachers over time without severe forgetting. Table 7 shows that after the model trains sequentially and finishes the Electronic Assembly environment, its Low difficulty AVG@8 score remains high at 91.88 for the Fantasy Alchemy domain and 88.98 for the Livestock Farming domain,

Table 7: Ensemble main table for No Shared Planning Patterns and Planning Patterns Conflict.

Pattern Mix	Model	Low				Middle				High				Avg	
		FA	LF	EA	Mean	FA	LF	EA	Mean	FA	LF	EA	Mean	L/M/H Mean	
AVG@8															
Teacher	Teacher FA	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	33.33	
	Teacher LF	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	33.33	
	Teacher EA	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	33.33	
4 Opt.:4 Sub.(c)	Instruct	54.77	49.53	55.31	53.20	0.31	0.23	0.62	0.39	0.00	0.00	0.00	0.00	17.86	
	After FA	95.23	64.61	35.47	65.10	70.62	2.89	0.00	24.51	55.16	0.00	0.00	18.39	36.00	
	After LF	93.75	88.28	19.14	67.06	63.28	61.48	0.00	41.59	48.44	30.55	0.00	26.33	44.99	
	After EA	91.88	88.98	89.77	90.21	58.83	57.27	66.56	60.89	47.34	29.14	42.97	39.82	63.64	
4 Opt.:4 Sub.(d)	Instruct	48.52	46.09	51.48	48.70	0.08	0.31	0.00	0.13	0.00	0.00	0.00	0.00	16.28	
	After FA	94.14	72.50	18.98	61.88	63.20	23.36	0.08	28.88	38.75	4.69	0.00	14.48	35.08	
	After LF	90.00	84.69	18.59	64.43	55.55	59.77	0.00	38.44	33.59	28.52	0.00	20.70	41.19	
	After EA	15.62	83.83	87.58	62.34	0.00	53.83	55.39	36.41	0.00	24.69	32.58	19.09	39.28	
PASS@8															
Teacher	Teacher FA	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	100.00	0.00	0.00	33.33	33.33	
	Teacher LF	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	0.00	100.00	0.00	33.33	33.33	
	Teacher EA	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	0.00	0.00	100.00	33.33	33.33	
4 Opt.:4 Sub.(c)	Instruct	80.62	75.00	77.50	77.71	1.88	1.88	4.38	2.71	0.00	0.00	0.00	0.00	26.81	
	After FA	96.25	87.50	55.00	79.58	75.62	19.38	0.00	31.67	65.00	0.00	0.00	21.67	44.31	
	After LF	95.62	91.25	30.00	72.29	71.88	66.88	0.00	46.25	53.75	36.25	0.00	30.00	49.51	
	After EA	94.38	92.50	91.25	92.71	68.12	64.38	72.50	68.33	54.38	35.00	48.12	45.83	68.96	
4 Opt.:4 Sub.(d)	Instruct	65.62	64.38	66.88	65.62	0.62	1.88	0.00	0.83	0.00	0.00	0.00	0.00	22.15	
	After FA	95.62	91.88	28.75	72.08	71.88	56.25	0.62	42.92	45.00	26.88	0.00	23.96	46.32	
	After LF	93.12	88.12	28.75	70.00	67.50	63.75	0.00	43.75	40.62	35.00	0.00	25.21	46.32	
	After EA	25.62	87.50	89.38	67.50	0.00	61.88	63.12	41.67	0.00	32.50	40.00	24.17	44.44	

**Figure 20:** Planning pattern distributions for 4 Opt.:4 Sub. (c) before and after multi-teacher OPD.

while it achieves 89.77 on the new Electronic Assembly domain. Figure 20 explains this stability. The agent learns to avoid domain specific traps and converges to the shared T1 Goal pattern with a 1.00 probability across all three domains. This shared compatible behavior preserves previous abilities and prevents conflicts between environments. (2) **Incompatible patterns cause severe conflicts.** Under the Form d setting, removing the shared T1 normal template forces the model into conflicts where new learning overwrites older behaviors. Table 7 highlights a massive performance drop. After the model learns from the Electronic Assembly teacher, its Low difficulty AVG@8 score on the initial Livestock Farming domain crashes from 90.00 to just 15.62. Figure 21 shows the underlying reason. Without the shared

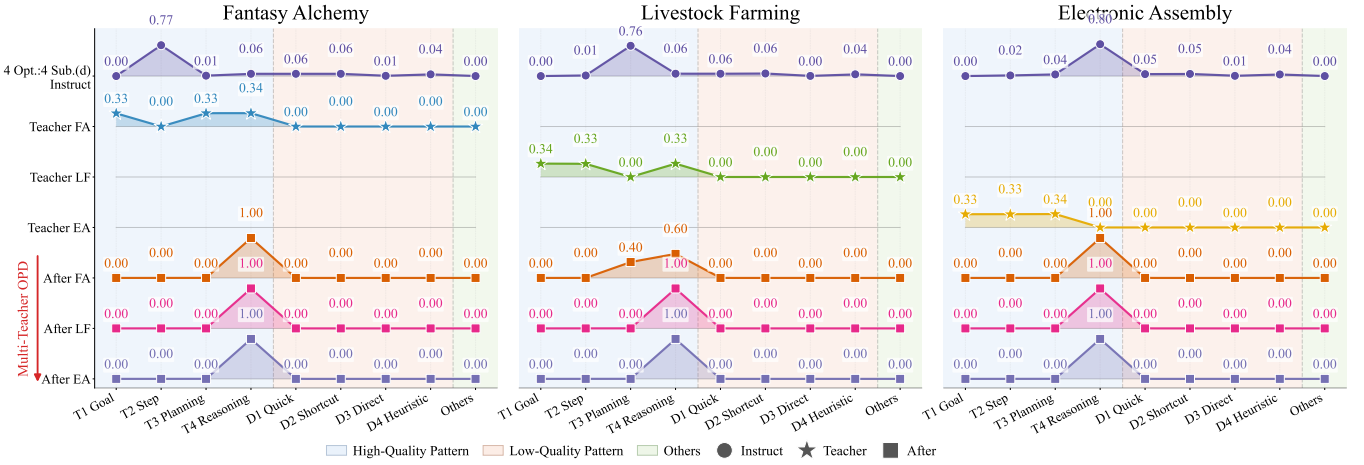


Figure 21: Planning pattern distributions for 4 Opt.:4 Sub. (d) before and after multi-teacher OPD.

pattern, the agent overfits to the new teacher and aggressively shifts to use a single conflicting pattern (T3 Planning) at a 1.00 probability everywhere. Because the remaining templates contain domain specific wrong shortcuts, applying one pattern universally triggers these traps and destroys the performance in previous domains like Fantasy Alchemy. The model has difficulty suppressing planning pattern 4, because the first two teachers already suppress patterns 2 and 3 to zero, while pattern 1 is not available during pretraining. This shows that the model learns cross-template knowledge transfer, but it also forgets much of its original knowledge.

Takeaway

Shared Planning Patterns with Planning Patterns Conflict still supports continual learning. MOPD learns from new teachers while preserving earlier domain skills when teacher patterns still share usable structure. **No Shared Planning Patterns with Planning Patterns Conflict causes severe forgetting.** Experts from different environments conflict with each other, so the student tends to forget one expert after learning another.

6.4. Parameter dynamics for MOPD

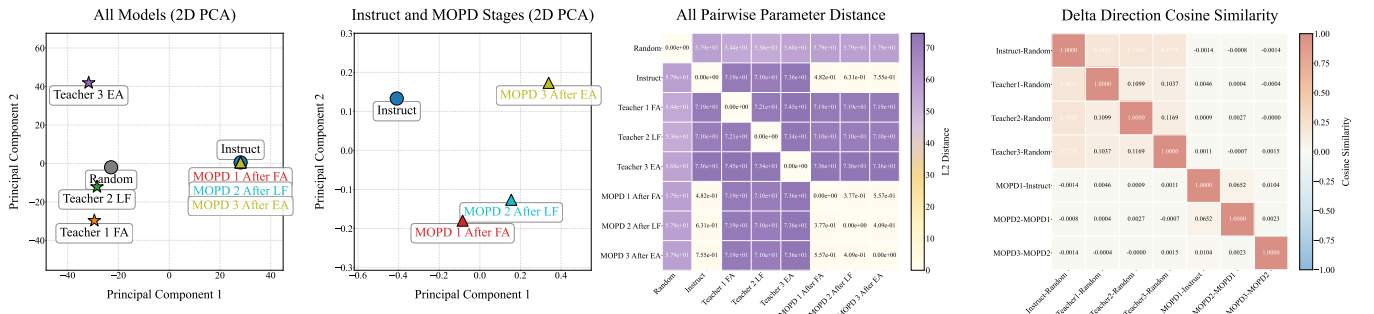


Figure 22: Parameter Dynamics for MOPD.

Task Setup. We study the parameter change of MOPD on three tasks, namely FA, LF, and EA. For each task, MOPD starts from the instruct model and then updates the model with the corresponding teacher signal. We compare Random, Instruct, three teacher models, and three MOPD stages. We use PCA to show the model positions in a low dimensional space. We also compute pairwise L2 distance between model parameters and cosine similarity between update directions.

Results. As shown in Figure 22. (1) **MOPD stays close to the instruct model.** In the PCA space of all models, Random and the three teacher models are far from Instruct. However, all MOPD stages stay near Instruct. The zoomed PCA plot

further shows that MOPD moves in a small local region after FA, LF, and EA. This means that MOPD does not copy the teacher parameters directly. Instead, it keeps the base model stable while learning useful task signals. (2) **MOPD makes small and stable updates.** The pairwise distance matrix shows that the distance between Instruct and MOPD 1 is only 4.82×10^{-1} , and the distances between two neighboring MOPD stages are also small. By contrast, the distances from Instruct to the teacher models are around 7.10×10^1 to 7.36×10^1 . This gap shows that MOPD changes the model with a much smaller parameter shift. The cosine similarity matrix also shows that the update directions of different MOPD stages have low similarity. Thus, each stage adds new task information without strongly repeating the previous update direction.

Takeaway

MOPD keeps the base model stable. The MOPD checkpoints remain close to the instruct model in both PCA space and parameter distance.

MOPD learns task signals with small updates. Each stage changes the parameters only slightly, and different stages bring different update directions.

7. Related Work

Long-Horizon Agentic Planning. As we are entering an excited era of long-horizon agents capable of pursuing goals over days and weeks, research in this field can be broadly categorized into two directions: benchmarks and applications. The benchmark direction includes representative efforts such as OSWorld 2.0 (Yuan et al., 2026), EdgeBench (Zhu et al., 2026a), DeepPlanning (Zhang et al., 2026b) and Long-Horizon-Terminal-Bench (Li et al., 2026f). For training, in terms of SFT data synthesis, some researchers study the synthesis of long-horizon trajectories, such as Agent-A1 (Bai et al., 2026) and Openresearcher (Li et al., 2026e). In terms of RL training algorithms, some researchers study training algorithms designed for long-horizon agents, such as SAO (Hou et al., 2026b) and TurnOPD (Zhou et al., 2026). However, existing large language models are pretrained on large-scale and non-transparent Internet corpora, making it difficult to identify the factors behind their capability improvement. Therefore, understanding how long-horizon planning ability develops remains challenging. With controlled environments and training data splits, our study provides a more controlled analysis of the factors that enhance long-horizon planning ability.

Single-Turn and Multi-Turn OPD. OPD provides a denser supervision signal than standard GRPO and receives increasing attention. Current research mainly focuses on two types of OPD: single-turn OPD and multi-turn OPD. For single-round OPD, early OPD is proposed as a knowledge distillation method (Gu et al., 2024, Agarwal et al., 2024, Lu and Lab, 2025). Recently, with the rise of RL and LLM capabilities (Song and Zheng, 2026), research directions are divided into three lines: strong-to-weak distillation (Li et al., 2026d, Fu et al., 2026, Jang et al., 2026, Li et al., 2026c, Yang et al., 2026b, Ko et al., 2026, Zhu et al., 2026b, Zhao et al., 2026a), self-distillation (Qu et al., 2026, Shenfeld et al., 2026, Hübotter et al., 2026, Zhao et al., 2026b, Zhang et al., 2026a, Li et al., 2026a, Yang et al., 2026a, He et al., 2026), and weak-to-strong distillation (Feng et al., 2026). Existing studies mainly focus on single-turn OPD, while multi-round OPD is rarely studied, and we are among the early works in this area. Existing studies mainly focus on post-training (Wang et al., 2026a, Zhou et al., 2026, Wang et al., 2026c), while opaque pre-training data limits the analysis of underlying mechanisms. We provide a unified study from pre-training to single-expert and multi-teacher distillation, covering data, algorithms, and parameter mechanisms.

Multi-Teacher OPD. Multi-Teacher OPD, as a model merging technique (or continual learning), plays an important role in the development of foundation models. The related research mainly focuses on two aspects: applications in different foundation models and algorithm optimization and improvement. In applications across different base models, there are mainly two uses. The first is cross-domain capability integration, including MiMo-V2-Flash (Xiao et al., 2026) and DeepSeek-V4 (Xu et al., 2026). The second is cross-stage capability retention, including GLM-5 (Zeng et al., 2026) and Nemotron-Cascade 2 (Yang et al., 2026c). Some other studies explore variants of multi-expert OPD, including CoPD (Gu et al., 2026), MAD-OPD (Wang et al., 2026b) and Uni-OPD (Hou et al., 2026a). However, existing training lacks a systematic analysis of the conditions that determine when model merging is effective or ineffective, limiting its practical applicability. Our work characterizes the effective boundaries of model merging through rigorous and controlled experimental settings with pre-trained models.

World Modeling and Compositional Generalization. For long-horizon planning, two abilities are especially important: World Modeling and Compositional Generalization. For world modeling, some works study whether internalizing a world model into existing language models can improve planning (Zhang et al., 2025b, Chen et al., 2025). In terms of compositional generalization, some works study whether language models can learn atomic abilities and spontaneously combine them into long-horizon planning abilities (Yuan et al., 2025, Men et al., 2026, Zhang et al., 2025a). However, these works cannot clearly determine whether the world modeling ability and compositional generalization ability already exist in pre-training and are only activated by post-training, or whether post-training can spontaneously develop abilities that do not exist in pre-training. Our work studies these questions from the pre-training stage with clean data, which allows us to clearly analyze the origin of these abilities.

8. Conclusion

This paper explores the mechanisms of long horizon planning in foundation models. We build a controllable synthetic environment to observe the training process across three stages. During pretraining, internal world models improve generalization. However, models require explicit long horizon data due to a lack of compositional generalization, and imperfect data harms performance. In the single expert stage, on policy distillation shows larger boundaries than GRPO and effectively optimizes low mutual information patterns. Conversely, forcing the model to distill unseen multi path knowledge destroys its planning abilities. Finally, in the multi expert stage, the model relies on shared distributions among teachers. Partially shared structures support skill transfer and continual learning, whereas completely conflicting patterns cause severe forgetting and a loss of cross environment capabilities. These findings provide empirical observations on how foundation models develop planning skills.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, pages 21246–21263, 2024.
- Lei Bai, Zongsheng Cao, Yang Chen, Zhiyao Cui, Shangheng Du, Yue Fan, Shiyang Feng, Zijie Guo, Haonan He, Liang He, et al. Scaling the horizon, not the parameters: Reaching trillion-parameter performance with a 35b agent. *arXiv preprint arXiv:2606.30616*, 2026.
- Pengfei Cao, Tianyi Men, Wencan Liu, Jingwen Zhang, Xuzhao Li, Xixun Lin, Dianbo Sui, Yanan Cao, Kang Liu, and Jun Zhao. Large language models for planning: A comprehensive and systematic survey. *arXiv preprint arXiv:2505.19683*, 2025.
- Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Kangrui Wang, Siyang Gao, Teng Xiao, Yee Whye Teh, Junxian He, and Manling Li. Internalizing world models via self-play finetuning for agentic rl. *arXiv preprint arXiv:2510.15047*, 2025.
- Shiyuan Feng, Huan-ang Gao, Haohan Chi, Hanlin Wu, Zhilong Zhang, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Weak-to-strong generalization via direct on-policy distillation. *arXiv preprint arXiv:2607.05394*, 2026.
- Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Jiakai Liu, Zhuo Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting on-policy distillation: Empirical failure modes and simple fixes. *arXiv preprint arXiv:2603.25562*, 2026.
- Naibin Gu, Chenxu Yang, Qingyi Si, Chuanyu Qin, Dingyu Yao, Peng Fu, Zheng Lin, Weiping Wang, Nan Duan, and Jiaqi Wang. Co-evolving policy distillation. *arXiv preprint arXiv:2604.27083*, 2026.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *International Conference on Learning Representations*, volume 2024, pages 32694–32717, 2024.
- Yinghui He, Simran Kaur, Adithya Bhaskar, Yongjin Yang, Jiarui Liu, Narutatsu Ri, Liam Fowl, Abhishek Panigrahi, Danqi Chen, and Sanjeev Arora. Self-distillation zero: Self-revision turns binary rewards into dense supervision. *arXiv preprint arXiv:2604.12002*, 2026.
- Wenjin Hou, Shangpin Peng, Weinong Wang, Zheng Ruan, Yue Zhang, Zhenglin Zhou, Mingqi Gao, Yifei Chen, Kaiqi Wang, Hongming Yang, et al. Uni-opd: Unifying on-policy distillation with a dual-perspective recipe. *arXiv preprint arXiv:2605.03677*, 2026a.
- Zhenyu Hou, Yujiang Li, Jie Tang, and Yuxiao Dong. Single-rollout asynchronous optimization for agentic reinforcement learning. *arXiv preprint arXiv:2607.07508*, 2026b.
- Jonas Hübner, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggyu Lim, and Taesup Kim. Stable on-policy distillation through adaptive target reformulation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 42217–42227, 2026.
- Jongwoo Ko, Sara Abdali, Young Jin Kim, Tianyi Chen, and Pashmina Cameron. Scaling reasoning efficiently via relaxed on-policy distillation. *arXiv preprint arXiv:2603.11137*, 2026.
- Gengsheng Li, Tianyu Yang, Junfeng Fang, Mingyang Song, Mao Zheng, Haiyun Guo, Dan Zhang, Jinqiao Wang, and Tat-Seng Chua. Unifying group-relative and self-distillation policy optimization via sample routing. *arXiv preprint arXiv:2604.02288*, 2026a.
- Jiachun Li, Zhuoran Jin, Tianyi Men, Yupu Hao, Kejian Zhu, Lingshuai Wang, Dongqi Huang, Longxiang Wang, Shengjia Hua, Lu Wang, et al. Agentic environment engineering for large language models: A survey of environment modeling, synthesis, evaluation, and application. *arXiv preprint arXiv:2606.12191*, 2026b.

- Jiaze Li, Hao Yin, Haoran Xu, Boshen Xu, Wenhui Tan, Zewen He, Jianzhong Ju, Zhenbo Luo, and Jian Luan. Video-opd: Efficient post-training of multimodal large language models for temporal video grounding via on-policy distillation. *arXiv preprint arXiv:2602.02994*, 2026c.
- Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, et al. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026d.
- Zhuofeng Li, Dongfu Jiang, Xueguang Ma, Haoxiang Zhang, Ping Nie, Yuyu Zhang, Kai Zou, Jianwen Xie, Yu Zhang, and Wenhui Chen. Openresearcher: A fully open pipeline for long-horizon deep research trajectory synthesis. *arXiv preprint arXiv:2603.20278*, 2026e.
- Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, et al. Long-horizon-terminal-bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading. *arXiv preprint arXiv:2607.08964*, 2026f.
- Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20251026. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- Tianyi Men, Zhuoran Jin, Pengfei Cao, Yubo Chen, Kang Liu, and Jun Zhao. Empowering gui agents via autonomous experience exploration and hindsight experience utilization for task planning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 36090–36108, 2026.
- Yuxiao Qu, Amrith Setlur, Virginia Smith, Ruslan Salakhutdinov, and Aviral Kumar. Pope: Learning to reason on hard problems via privileged on-policy exploration. *arXiv preprint arXiv:2601.18779*, 2026.
- Idan Shenfeld, Mehul Damani, Jonas Hübötter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.
- Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.
- Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, et al. Skill-sd: Skill-conditioned self-distillation for multi-turn llm agents. *arXiv preprint arXiv:2604.10674*, 2026a.
- Jianze Wang, Ying Liu, Jinlong Chen, Xuchun Hu, Qilong Zhang, Yu Cao, Jun Wang, Hua Yang, Yong Xie, and Qianglong Chen. Mad-opd: Breaking the ceiling in on-policy distillation via multi-agent debate. *arXiv preprint arXiv:2605.01347*, 2026b.
- Jiaqi Wang, Wenhao Zhang, Weijie Shi, Yaliang Li, and James Cheng. Tcod: Exploring temporal curriculum in on-policy distillation for multi-turn autonomous agents. *arXiv preprint arXiv:2604.24005*, 2026c.
- Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.
- Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-distilled rlvr. *arXiv preprint arXiv:2604.03128*, 2026a.
- Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026b.
- Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, et al. Nemotron-cascade 2: Post-training llms with cascade rl and multi-domain on-policy distillation. *arXiv preprint arXiv:2603.19220*, 2026c.

- Lifan Yuan, Weize Chen, Yuchen Zhang, Ganqu Cui, Hanbin Wang, Ziming You, Ning Ding, Zhiyuan Liu, Maosong Sun, and Hao Peng. From $f(x)$ and $g(x)$ to $f(g(x))$: Llms learn new skills in rl by composing old ones. *arXiv preprint arXiv:2509.25123*, 2025.
- Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, Weiming Wu, Jiayang Sun, Jiamin Song, Kaiqian Cui, Bowen Wang, Haoyuan Wu, Yitong Li, et al. Osworld2. 0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026.
- Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- Charlie Zhang, Graham Neubig, and Xiang Yue. On the interplay of pre-training, mid-training, and rl on reasoning language models. *arXiv preprint arXiv:2512.07783*, 2025a.
- Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, et al. Agent learning via early experience. *arXiv preprint arXiv:2510.08558*, 2025b.
- Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation. *arXiv preprint arXiv:2604.01193*, 2026a.
- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints. *arXiv preprint arXiv:2601.18137*, 2026b.
- Anhao Zhao, Haoran Xin, Yingqi Fan, Junlong Tong, Wenjie Li, and Xiaoyu Shen. Decoupling kl and trajectories: A unified perspective for sft, dagger, offline rl, and opd in llm distillation. *arXiv preprint arXiv:2605.16826*, 2026a.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026b.
- Yuhang Zhou, Kai Zheng, Haoling Li, Dengyun Peng, Can Xu, and Jingjing Chen. Turnopd: Making on-policy distillation turn-aware for efficient long-horizon agent training. *arXiv preprint arXiv:2607.05804*, 2026.
- Deyao Zhu, Xin Zhou, Shengling Qin, Xuekai Zhu, Hangliang Ding, Shu Zhong, Zixin Wen, Zhonglin Xie, Chenhui Gou, Linxuan Ren, et al. Edgebench: Unveiling scaling laws of learning from real-world environments. *arXiv preprint arXiv:2607.05155*, 2026a.
- Siqi Zhu, Xuyan Ye, Hongyu Lu, Weiye Shi, and Ge Liu. The many faces of on-policy distillation: Pitfalls, mechanisms, and fixes. *arXiv preprint arXiv:2605.11182*, 2026b.

Appendix Table of Contents

A Pre-training Details	37
A.1 Pre-trained Model Configuration	37
A.2 Pre-training Schema Configuration	37
A.3 Pre-training Corpus Configuration	38
A.4 Pre-training Configuration	40

A. Pre-training Details

A.1. Pre-trained Model Configuration

This section details the specific architectural design and tokenizer configuration for the randomly initialized language model utilized in our study. The model is built following the Qwen2ForCausalLM architecture specifications. In total, this configuration yields a compact model with approximately 100 million (100M) parameters. We call it Qwen2.5-100M. Table 8 lists the detailed network hyperparameters, and Table 9 summarizes the configuration properties of the accompanying byte-level BPE tokenizer. Finally, it should be noted that the tokenizer is trained on the corresponding full pre-training corpus. Because this corpus varies slightly across different chapters, we perform tokenization separately for each. This variation does not affect our overall experimental conclusions.

Table 8: Pre-trained model architectural hyperparameters.

Hyperparameter	Value
Model Type	qwen2
Architecture	Qwen2ForCausalLM
Number of Hidden Layers	12
Hidden Size (d_{model})	768
Intermediate Size (d_{ff})	3072
Number of Attention Heads (n_{head})	12
Number of Key-Value Heads ($n_{\text{kv_head}}$)	2 (Grouped-Query Attention)
Attention Head Dimension	64
Hidden Activation Function	SiLU
Attention Dropout	0.0
RMS Norm Epsilon (ϵ)	1×10^{-6}
RoPE Theta (θ)	1,000,000.0
Max Position Embeddings	2048
Vocabulary Size (V)	3000
Tie Word Embeddings	True
Data Type (dtype)	bfloat16

A.2. Pre-training Schema Configuration

The pretraining corpus is constructed over three distinct domains, namely Fantasy Alchemy, Livestock Farming, and Electronic Assembly. As summarized in Table 10, each domain contains 200 item nodes and a total of 4000 item names. The task space is organized into 5 hierarchical levels, with each level comprising 40 item categories and 20 instances per category.

Each item category is defined by a compositional schema, where higher-level items are recursively generated from lower-level components through logical conjunction (AND) and disjunction (OR) operators. In particular, the input

Table 9: Tokenizer configuration and training details.

Tokenizer Property	Value
Tokenizer Type	byte_level_bpe+regex
Vocabulary Size	3000
Minimum Token Frequency	2
Limit Alphabet	1000
Maximum Token Length	100
Byte-level Add Prefix Space	False
BOS Token	[BOS]
EOS Token	[EOS]
Special Tokens	[UNK], [PAD], [BOS], [EOS], <question>, </question>, <solution>, </solution>, <answer>, </answer>

Table 10: Pretraining schema statistics across three domains.

Metric	Fantasy Alchemy	Livestock Farming	Electronic Assembly
Total nodes	200	200	200
Total item names	4000	4000	4000
One-input ratio	30.00%	30.00%	30.00%
Two-input ratio	70.00%	70.00%	70.00%
Avg atoms for target L2 from source L1	1.82	1.82	1.82
Avg atoms for target L3 from source L1	3.08	3.08	3.08
Avg atoms for target L3 from source L2	1.68	1.68	1.68
Avg atoms for target L4 from source L1	4.17	4.17	4.17
Avg atoms for target L4 from source L2	2.23	2.23	2.23
Avg atoms for target L4 from source L3	1.65	1.65	1.65
Avg atoms for target L5 from source L1	5.78	5.78	5.78
Avg atoms for target L5 from source L2	3.15	3.15	3.15
Avg atoms for target L5 from source L3	2.38	2.38	2.38
Avg atoms for target L5 from source L4	1.65	1.65	1.65

degree distribution is controlled to maintain a fixed structural prior, with 30% of items following a one-input dependency and 70% requiring two-input compositions. This induces a sparse but non-trivial dependency graph over items.

The compositional depth increases with level, leading to progressively more complex construction paths from base materials. As shown in Table 10, the expected number of atomic components required to construct target items grows with hierarchy depth, e.g., from Level 1 to Level 2 requiring on average 1.82 atoms, while Level 5 items require up to 5.78 atoms when traced back to Level 1 sources.

To further increase task difficulty and robustness, each planning instance is constructed by mixing target-relevant items with additional noise items, preventing trivial pattern matching and encouraging the model to recover correct compositional structure under partial observability. The resulting environment thus provides a controlled benchmark for evaluating long-horizon compositional planning under structured but noisy generation conditions.

A.3. Pre-training Corpus Configuration

We construct the datasets from three synthetic domains: electronic assembly, livestock farming, and fantasy alchemy. For each domain, the dataset builder reads a shared item configuration, a list of concrete instantiations, and two recipe schemas. The two schemas define two planning environments, denoted as split A and split B. Each instantiation maps every abstract item node to one concrete item name, so split A and split B share the same item names but use different recipe graphs. We adopt a counterfactual formulation of the state transition function for scalability. This reformulation follows the same environment transition rules and produces the same observable interaction outcomes, so it does not affect the empirical results.

For each concrete instantiation, the builder enumerates every non base target item and every feasible starting layer below the target layer. It recursively collects the materials that are required by the A recipe graph and the B recipe graph at the selected starting layer. The initial inventory contains the union of these required materials, together with two to four distractor items sampled from the same starting layer. The target item, the input inventory, the required materials, the distractors, and the gold plan are then stored as one sample. The gold plan is obtained by recursively expanding the target item according to the corresponding recipe graph until all required intermediate items are produced. The number of actions in this gold plan defines the field total step nums.

The first 20 concrete instantiations form the test pool. Instantiations with indices from 20 to 219 form the post training pool. All remaining instantiations form the pretraining pool. The post training and test samples are further grouped by plan length. The low group contains samples with one to five steps, the mid group contains samples with six to eight steps, and the high group contains samples with at least nine steps. The teacher_all pretraining set contains all generated samples for a split and a domain, including samples from all instantiation ranges, and serves as the full teacher data source.

As shown in Table 11. Table rows follow the file naming convention used by the dataset builder. The row label Category gives the synthetic domain. The row label Split summarizes the recipe split and dataset subset. The token A or B identifies which recipe graph produces the gold trajectory. The token pretrain denotes samples from the pretraining pool, posttrain denotes samples from the post training pool, and test denotes held out evaluation samples. The token teacher_all denotes the full teacher data source for the corresponding split and domain with both train and test datasets. The tokens low, mid, and high denote the step based difficulty groups. The token final, when present, denotes the final evaluation copy of the corresponding test subset. The column Samples reports the number of samples in the row. The columns Avg, Min, and Max report the mean, minimum, and maximum gold plan length. The columns named Step report the number of samples with each exact gold plan length.

Category	Split	Samples	Avg	Min	Max	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6	Step 7	Step 8	Step 9	Step 10	Step 11	Step 12	Step 13
Pretrain																		
electronic assembly	A / pretrain	120000	3.00	1	12	48000	12300	26400	5700	8100	10200	1500	2100	2100	3000	300	300	0
	A / pretrain / teacher_all	208000	3.00	1	12	83200	21320	45760	9880	14040	17680	2600	3640	3640	5200	520	520	0
	B / pretrain	120000	3.01	1	13	48000	11700	27300	6600	6900	8100	4500	1200	2400	1500	900	600	300
	B / pretrain / teacher_all	208000	3.01	1	13	83200	20280	47320	11440	11960	14040	7800	2080	4160	2600	1560	1040	520
fantasy alchemy	A / pretrain	120000	3.00	1	12	48000	12300	26400	5700	8100	10200	1500	2100	2100	3000	300	300	0
	A / pretrain / teacher_all	208000	3.00	1	12	83200	21320	45760	9880	14040	17680	2600	3640	3640	5200	520	520	0
	B / pretrain	120000	3.01	1	13	48000	11700	27300	6600	6900	8100	4500	1200	2400	1500	900	600	300
	B / pretrain / teacher_all	208000	3.01	1	13	83200	20280	47320	11440	11960	14040	7800	2080	4160	2600	1560	1040	520
livestock farming	A / pretrain	120000	3.00	1	12	48000	12300	26400	5700	8100	10200	1500	2100	2100	3000	300	300	0
	A / pretrain / teacher_all	208000	3.00	1	12	83200	21320	45760	9880	14040	17680	2600	3640	3640	5200	520	520	0
	B / pretrain	120000	3.01	1	13	48000	11700	27300	6600	6900	8100	4500	1200	2400	1500	900	600	300
	B / pretrain / teacher_all	208000	3.01	1	13	83200	20280	47320	11440	11960	14040	7800	2080	4160	2600	1560	1040	520
Posttrain																		
electronic assembly	A / low	67000	2.14	1	5	32000	8200	17600	3800	5400	0	0	0	0	0	0	0	0
	B / low	67000	2.13	1	5	32000	7800	18200	4400	4600	0	0	0	0	0	0	0	0
	A / mid	9200	6.41	6	8	0	0	0	0	0	6800	1000	1400	0	0	0	0	0
	B / mid	9200	6.50	6	8	0	0	0	0	0	5400	3000	800	0	0	0	0	0
	A / high	3800	9.79	9	12	0	0	0	0	0	0	0	0	1400	2000	200	200	0
	B / high	3800	10.11	9	13	0	0	0	0	0	0	0	0	1600	1000	600	400	200
fantasy alchemy	A / low	67000	2.14	1	5	32000	8200	17600	3800	5400	0	0	0	0	0	0	0	0
	B / low	67000	2.13	1	5	32000	7800	18200	4400	4600	0	0	0	0	0	0	0	0
	A / mid	9200	6.41	6	8	0	0	0	0	0	6800	1000	1400	0	0	0	0	0
	B / mid	9200	6.50	6	8	0	0	0	0	0	5400	3000	800	0	0	0	0	0
	A / high	3800	9.79	9	12	0	0	0	0	0	0	0	0	1400	2000	200	200	0
	B / high	3800	10.11	9	13	0	0	0	0	0	0	0	0	1600	1000	600	400	200
livestock farming	A / low	67000	2.14	1	5	32000	8200	17600	3800	5400	0	0	0	0	0	0	0	0
	B / low	67000	2.13	1	5	32000	7800	18200	4400	4600	0	0	0	0	0	0	0	0
	A / mid	9200	6.41	6	8	0	0	0	0	0	6800	1000	1400	0	0	0	0	0
	B / mid	9200	6.50	6	8	0	0	0	0	0	5400	3000	800	0	0	0	0	0
	A / high	3800	9.79	9	12	0	0	0	0	0	0	0	0	1400	2000	200	200	0
	B / high	3800	10.11	9	13	0	0	0	0	0	0	0	0	1600	1000	600	400	200
Test																		
electronic assembly	A / low	6700	2.14	1	5	3200	820	1760	380	540	0	0	0	0	0	0	0	0
	B / low	6700	2.13	1	5	3200	780	1820	440	460	0	0	0	0	0	0	0	0
	A / mid	920	6.41	6	8	0	0	0	0	0	680	100	140	0	0	0	0	0
	B / mid	920	6.50	6	8	0	0	0	0	0	540	300	80	0	0	0	0	0
	A / high	380	9.79	9	12	0	0	0	0	0	0	0	0	140	200	20	20	0
	B / high	380	10.11	9	13	0	0	0	0	0	0	0	0	160	100	60	40	20
fantasy alchemy	A / low	6700	2.14	1	5	3200	820	1760	380	540	0	0	0	0	0	0	0	0
	B / low	6700	2.13	1	5	3200	780	1820	440	460	0	0	0	0	0	0	0	0
	A / mid	920	6.41	6	8	0	0	0	0	0	680	100	140	0	0	0	0	0
	B / mid	920	6.50	6	8	0	0	0	0	0	540	300	80	0	0	0	0	0
	A / high	380	9.79	9	12	0	0	0	0	0	0	0	0	140	200	20	20	0
	B / high	380	10.11	9	13	0	0	0	0	0	0	0	0	160	100	60	40	20
livestock farming	A / low	6700	2.14	1	5	3200	820	1760	380	540	0	0	0	0	0	0	0	0
	B / low	6700	2.13	1	5	3200	780	1820	440	460	0	0	0	0	0	0	0	0
	A / mid	920	6.41	6	8	0	0	0	0	0	680	100	140	0	0	0	0	0
	B / mid	920	6.50	6	8	0	0	0	0	0	540	300	80	0	0	0	0	0
	A / high	380	9.79	9	12	0	0	0	0	0	0	0	0	140	200	20	20	0
	B / high	380	10.11	9	13	0	0	0	0	0	0	0	0	160	100	60	40	20

Table 11: Statistics of pretrain, posttrain, and test datasets.

A.4. Pre-training Configuration

As summarized in Table 12, our SFT corpus consists of student and teacher datasets with distinct characteristics. The teacher datasets act as domain experts, encompassing both the training and post-training sets. Each teacher dataset contains exactly 622,960 samples, specializing exclusively in one of three fields (fantasy alchemy, livestock farming, and electronic assembly) while covering three difficulty levels. In contrast, the student datasets broadly encompass all three domains and are constructed using specific sampling ratios across the three difficulty tiers (low, mid, and high), as reflected in their dataset names, yielding sample sizes that scale from roughly 322K to 1.07M. Additionally, both student and teacher datasets include subsets with and without CoT reasoning. Token statistics highlight that CoT subsets produce significantly longer outputs (averaging 186-230 tokens) compared to the direct answers in non-CoT subsets (averaging ~ 30 tokens). Meanwhile, the average input lengths remain relatively stable across all datasets,

Table 12: Estimated token statistics of SFT datasets.

Dataset	#Samples	Input Tokens	Output Tokens	Avg Input	Avg Output
Student CoT, low50% mid0% high0%	322,711	85,921,610	60,195,896	266.25	186.53
Student CoT, low100% mid0% high0%	645,300	170,851,435	120,079,424	264.76	186.08
Student CoT, low100% mid5% high0%	658,591	176,507,591	123,342,635	268.01	187.28
Student CoT, low100% mid50% high0%	778,129	220,910,979	153,718,139	283.90	197.55
Student CoT, low100% mid100% high0%	910,800	272,430,026	189,074,794	299.11	207.59
Student CoT, low100% mid100% high5%	919,146	277,076,378	191,666,574	301.45	208.53
Student CoT, low100% mid100% high50%	994,512	310,958,940	215,904,279	312.67	217.10
Student CoT, low100% mid100% high100%	1,078,200	349,875,900	240,656,936	324.50	223.20
Student w/o CoT, low100% mid100% high100%	1,078,200	350,036,228	32,182,761	324.65	29.85
Teacher CoT, Fantasy Alchemy	622,960	211,826,833	142,973,369	340.03	229.51
Teacher CoT, Livestock Farming	622,960	201,144,813	140,325,789	322.89	225.26
Teacher CoT, Electronic Assembly	622,960	192,900,810	133,524,250	309.65	214.34
Teacher w/o CoT, Fantasy Alchemy	622,960	211,628,047	18,799,998	339.71	30.18
Teacher w/o CoT, Livestock Farming	622,960	199,517,081	18,476,308	320.27	29.66
Teacher w/o CoT, Electronic Assembly	622,960	193,367,781	18,393,766	310.40	29.53

ranging from 264 to 340 tokens.

All experiments are conducted with 2 GPUs training and share the same optimization setup: a per-device training batch size of 128, 2 gradient accumulation steps, a learning rate of $1.0\text{e-}4$ with a cosine scheduler, a training steps of 9000, a warmup ratio of 0.1, and bf16 precision enabled. We use full-parameter SFT on the same base model (*Qwen2.5-100M*).